

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC [TÊN TRƯỜNG]
KHOA [TÊN KHOA]**

BÁO CÁO ĐỒ ÁN

Đề tài:

THIẾT KẾ VÀ XÂY DỰNG TRÒ CHƠI GIẢI ĐỒ BLOCKJAM3D TRÊN NỀN TẢNG MOBILE

Giảng viên hướng dẫn: [GVHD]

Sinh viên thực hiện: [Họ tên SV]

Mã sinh viên: [MSSV]

Lớp: [Tên lớp]

Khóa: [Khóa học]

[Địa điểm] – Năm 2026

LỜI CẢM ƠN

Trong suốt quá trình học tập và thực hiện đồ án này, em đã nhận được rất nhiều sự giúp đỡ, hướng dẫn quý báu từ thầy cô, gia đình và bạn bè. Lời đầu tiên, em xin gửi lời cảm ơn chân thành và sâu sắc nhất đến Ban giám hiệu Trường Đại học [TÊN TRƯỜNG], Ban chủ nhiệm Khoa [TÊN KHOA] cùng toàn thể quý thầy cô trong khoa đã tận tâm truyền đạt cho em những kiến thức nền tảng vô cùng quý giá trong suốt những năm học vừa qua. Những kiến thức đó không chỉ là tiền đề giúp em hoàn thành báo cáo này, mà còn là hành trang vững chắc để em bước vào con đường sự nghiệp sau khi tốt nghiệp.

Đặc biệt, em xin được bày tỏ lòng biết ơn sâu sắc nhất đến thầy/cô [GVHD] – người đã trực tiếp hướng dẫn, định hướng và tận tình chỉ bảo em trong suốt quá trình thực hiện đồ án. Sự nhiệt huyết, kiên nhẫn và những lời khuyên quý báu của thầy/cô không chỉ giúp em tháo gỡ những khó khăn về mặt chuyên môn, mà còn truyền cảm hứng để em hoàn thiện bản thân trong tư duy nghiên cứu và phát triển phần mềm. Mỗi buổi gặp gỡ, mỗi lần phản biện đều mang lại cho em nhiều bài học vô cùng quý giá.

Em cũng xin gửi lời cảm ơn tới các bạn cùng lớp, cùng nhóm đã luôn đồng hành, hỗ trợ và chia sẻ kinh nghiệm trong suốt quá trình học tập và thực hiện đồ án. Sự góp ý chân thành và sự động viên kịp thời từ các bạn đã giúp em vượt qua nhiều giai đoạn khó khăn để hoàn thiện sản phẩm hiện tại.

Cuối cùng, em xin gửi lời tri ân sâu sắc đến gia đình – những người đã luôn yêu thương, ủng hộ và tạo mọi điều kiện thuận lợi nhất để em yên tâm học tập và nghiên cứu. Đó là động lực lớn nhất giúp em hoàn thành đồ án này.

Do giới hạn về thời gian, kiến thức và kinh nghiệm thực tiễn, báo cáo đồ án không thể tránh khỏi những thiếu sót. Em rất mong nhận được sự góp ý chân thành từ quý thầy cô và các bạn để báo cáo được hoàn thiện hơn, đồng thời giúp em rút ra những bài học cần thiết cho công việc trong tương lai.

Em xin chân thành cảm ơn!

[Địa điểm], ngày ... tháng ... năm 2026

Sinh viên thực hiện

(Ký và ghi rõ họ tên)

[Họ tên SV]

MỤC LỤC

DANH MỤC TỪ VIẾT TẮT VÀ THUẬT NGỮ

STT	Từ viết tắt / Thuật ngữ	Diễn giải
1	Unity	Game engine đa nền tảng do Unity Technologies phát triển, dùng cho phát triển game 2D/3D.
2	URP	Universal Render Pipeline – pipeline render đa nền tảng tối ưu cho mobile của Unity.
3	C#	C-Sharp – ngôn ngữ lập trình hướng đối tượng do Microsoft phát triển, ngôn ngữ scripting chính của Unity.
4	GameObject	Đối tượng cơ bản trong Unity – có thể chứa nhiều Component.
5	Prefab	Mẫu (template) GameObject đã được cấu hình sẵn, có thể được tạo bản sao tại runtime.
6	ScriptableObject (SO)	Đối tượng dữ liệu dạng asset trong Unity, dùng để lưu cấu hình tách rời khỏi GameObject.
7	Addressables	Hệ thống quản lý và load tài nguyên động theo address/label của Unity.
8	Singleton	Mẫu thiết kế (design pattern) đảm bảo chỉ có duy nhất một thể hiện (instance) của một lớp.
9	DDOL	Don't Destroy On Load – đối tượng tồn tại xuyên các scene trong Unity.
10	BFS	Breadth-First Search – thuật toán tìm kiếm theo chiều rộng trên đồ thị.
11	DOTween	Thư viện tween animation phổ biến cho Unity, được phát hành bởi Demigiant.
12	Firestore	Cloud Firestore – CSDL NoSQL của Google Firebase.
13	RTDB	Realtime Database – CSDL realtime của Google Firebase, dùng cho thông báo tức thời.
14	SDK	Software Development Kit – bộ công cụ phát triển phần mềm.
15	API	Application Programming Interface – giao diện lập trình ứng dụng.
16	UI	User Interface – giao diện người dùng.

STT	Từ viết tắt / Thuật ngữ	Diễn giải
17	UX	User Experience – trải nghiệm người dùng.
18	FPS	Frames Per Second – số khung hình hiển thị trên giây.
19	SFX	Sound Effect – hiệu ứng âm thanh.
20	EDM4U	External Dependency Manager for Unity – công cụ quản lý phụ thuộc native cho Unity.
21	IDE	Integrated Development Environment – môi trường phát triển tích hợp.
22	OOP	Object-Oriented Programming – lập trình hướng đối tượng.
23	JSON	JavaScript Object Notation – định dạng dữ liệu phổ biến.
24	UML	Unified Modeling Language – ngôn ngữ mô hình hóa thống nhất.
25	BFD	Business Function Diagram – sơ đồ chức năng nghiệp vụ.
26	DFD	Data Flow Diagram – sơ đồ luồng dữ liệu.
27	ERD	Entity Relationship Diagram – sơ đồ thực thể – liên kết.
28	Use Case	Trường hợp sử dụng – mô tả tương tác giữa actor và hệ thống.
29	GSP	GridSpot – máng/lỗ sinh khối block trong game.
30	Match-3	Cơ chế gameplay ghép 3 đối tượng cùng loại để tiêu diệt – nền tảng của thể loại match puzzle.

DANH MỤC BẢNG

- Bảng 1.1. So sánh các game engine phổ biến cho phát triển game mobile.
- Bảng 1.2. So sánh các phiên bản Unity LTS.
- Bảng 2.1. Bảng yêu cầu chức năng cho người chơi (Player).
- Bảng 2.2. Bảng yêu cầu chức năng cho người quản trị (Admin/Level Designer).
- Bảng 2.3. Bảng yêu cầu phi chức năng.
- Bảng 2.4. Bảng mô tả thực thể UserData.
- Bảng 2.5. Bảng mô tả thực thể LevelData.
- Bảng 2.6. Bảng mô tả thực thể Friend và FriendRequest.
- Bảng 2.7. Bảng mô tả Use Case 'Chơi một level'.
- Bảng 2.8. Bảng mô tả Use Case 'Sử dụng Booster'.
- Bảng 2.9. Bảng mô tả Use Case 'Đăng nhập Google'.
- Bảng 2.10. Bảng mô tả Use Case 'Gửi quà cho bạn bè'.
- Bảng 3.1. Bảng các Singleton Manager chính trong dự án.
- Bảng 3.2. Bảng các Event Action trong CustomeEventSystem.
- Bảng 3.3. Cấu trúc thư mục Assets/Scripts.
- Bảng 3.4. Bảng quy ước commit và branch git.
- Bảng 3.5. Bảng các module triển khai theo tuần.
- Bảng 3.6. Bảng các kịch bản kiểm thử chức năng (Functional Test).
- Bảng 3.7. Bảng các kịch bản kiểm thử hiệu năng (Performance Test).
- Bảng 3.8. Bảng kết quả khảo sát người dùng cuối.

DANH MỤC HÌNH

- Hình 1.1. Thị phần các game thể loại Match Puzzle trên thị trường mobile (minh họa).
- Hình 1.2. Giao diện một số game match-3 nổi tiếng (Candy Crush, Block Blast, Toon Blast).
- Hình 1.3. Kiến trúc tổng thể Unity Engine.
- Hình 1.4. Giao diện Unity Editor 6000.0.60f1.
- Hình 1.5. Quy trình phát triển phần mềm Agile/Scrum.
- Hình 2.1. Sơ đồ chức năng nghiệp vụ (BFD) tổng thể.
- Hình 2.2. Sơ đồ luồng dữ liệu mức ngữ cảnh (DFD level 0).
- Hình 2.3. Sơ đồ luồng dữ liệu mức 1 (DFD level 1).
- Hình 2.4. Sơ đồ thực thể – liên kết (ERD).
- Hình 2.5. Sơ đồ Use Case tổng thể.
- Hình 2.6. Sơ đồ Sequence – Chơi một round.
- Hình 2.7. Sơ đồ Sequence – Sử dụng booster Shuffle.
- Hình 2.8. Sơ đồ Sequence – Gửi yêu cầu kết bạn.
- Hình 2.9. Sơ đồ Class tổng thể của hệ thống.
- Hình 2.10. Class diagram chi tiết module LevelManager.
- Hình 3.1. Sơ đồ kiến trúc tổng thể hệ thống BlockJam3D.
- Hình 3.2. Sơ đồ Scene flow.
- Hình 3.3. Sơ đồ Event Bus.
- Hình 3.4. Cấu trúc thư mục dự án Unity.
- Hình 3.5. Bảng kanban quản lý tiến độ trên Trello/Jira.
- Hình 3.6. Giao diện công cụ Level Editor (Admin).
- Hình 3.7. Màn hình Main Menu (Khách).

Hình 3.8. Màn hình Gameplay.

Hình 3.9. Popup Settings và Add Heart.

Hình 3.10. Popup Win Game và Lose Game.

Hình 3.11. Màn hình Leaderboard – Tab Friends.

Hình 3.12. Màn hình Leaderboard – Tab Players.

Hình 3.13. Popup Send Gift cho bạn bè.

Hình 3.14. Biểu đồ FPS đo trên thiết bị Android tầm trung.

Hình 3.15. Kết quả kiểm thử bộ nhớ (Profiler).

MỞ ĐẦU

1. Lý do chọn đề tài

Trong những năm gần đây, ngành công nghiệp game đã có những bước phát triển vô cùng mạnh mẽ trên phạm vi toàn cầu. Báo cáo của Newzoo và DataAI chỉ ra rằng thị trường game di động (mobile game) chiếm hơn 50% doanh thu toàn ngành game và tiếp tục tăng trưởng ổn định trong những năm tới. Cùng với đó, sự phổ cập của các thiết bị di động thông minh đã biến game mobile trở thành hình thức giải trí phổ biến nhất mà bất kỳ ai cũng có thể tiếp cận, không phân biệt độ tuổi, giới tính hay nghề nghiệp.

Trong vô vàn thể loại game di động, dòng game giải đố casual – đặc biệt là các tựa game thuộc thể loại match-3 và sort puzzle – luôn duy trì sức hút mãnh liệt với người chơi. Các tựa game tiêu biểu như Candy Crush Saga, Royal Match, Block Blast, Toon Blast, Gardenscapes... không chỉ thành công về mặt doanh thu mà còn thể hiện một xu hướng thiết kế gameplay: cốt lõi đơn giản, dễ tiếp cận, nhưng cấp độ thử thách tăng dần và có chiều sâu, kết hợp với các tính năng xã hội (leaderboard, kết bạn, gửi quà, sự kiện) để giữ chân người chơi lâu dài.

Xuất phát từ sự yêu thích cá nhân với thể loại game này, đồng thời nhận thấy đây là một lĩnh vực còn nhiều dư địa cho các sản phẩm Việt Nam, em đã lựa chọn đề tài "Thiết kế và xây dựng trò chơi giải đố BlockJam3D trên nền tảng Mobile" làm đề tài đồ án. Quá trình thực hiện đồ án không chỉ là cơ hội để em vận dụng các kiến thức đã được trang bị về lập trình hướng đối tượng, cấu trúc dữ liệu và giải thuật, phân tích thiết kế hệ thống, mà còn là cơ hội để em tiếp cận thực tế với một game engine mạnh mẽ – Unity – cùng các công nghệ cloud hiện đại như Firebase Firestore và Realtime Database.

Bên cạnh đó, đề tài này còn đặt ra nhiều thách thức kỹ thuật thú vị: làm sao để tổ chức kiến trúc của một game có hàng chục thành phần tương tác lẫn nhau (board, cell play, booster, tutorial, save/load, leaderboard...) một cách rõ ràng, dễ mở rộng; làm sao để tối ưu hiệu năng giúp game chạy mượt mà ở 60 FPS trên các thiết bị Android tầm trung; làm sao thiết kế công cụ Level Editor giúp người làm nội dung tạo level dễ dàng mà không cần lập trình. Đây chính là cơ hội rất tốt để em rèn luyện kỹ năng giải quyết vấn đề thực tế.

2. Mục tiêu của đề tài

Đề tài hướng đến các mục tiêu cụ thể như sau:

- Nghiên cứu và làm chủ được các kiến thức cơ bản đến nâng cao về Unity Engine phiên bản 6000.0.60f1 (URP) và ngôn ngữ C#, áp dụng được các mẫu thiết kế (design patterns) phổ biến vào dự án thực tế.
- Tiến hành phân tích, thiết kế hệ thống một game match-3 hoàn chỉnh theo cả hướng cấu trúc (BFD, DFD, ERD) và hướng đối tượng (Use Case, Sequence, Class) để có cái nhìn đa chiều về kiến trúc phần mềm.
- Xây dựng được sản phẩm cuối là một game di động BlockJam3D hoàn chỉnh trên nền tảng Android, với 10 level (mỗi level gồm 3 round), 4 loại booster, hệ thống tutorial, save/load dữ liệu, leaderboard kết nối Firebase, hệ thống kết bạn và gửi quà tặng.
- Tích hợp cloud backend (Firebase Firestore + Realtime Database) phục vụ tính năng đồng bộ dữ liệu, leaderboard toàn cầu và tính năng xã hội (kết bạn, gửi quà, gửi tim).
- Phát triển công cụ Level Editor (Unity Editor extension) cho phép người làm nội dung dễ dàng thiết kế level mới mà không cần can thiệp source code.
- Đảm bảo hiệu năng game đạt tối thiểu 60 FPS ổn định trên thiết bị Android tầm trung (RAM 4GB, Snapdragon 6xx series hoặc tương đương), thời gian khởi động không quá 5 giây.
- Hoàn thiện bộ tài liệu thiết kế và báo cáo đồ án đầy đủ, làm cơ sở để mở rộng và bảo trì sản phẩm trong tương lai.

3. Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu: Trò chơi giải đố thể loại match-3 / sort puzzle trên nền tảng di động, kết hợp với hệ thống cloud backend cho các tính năng xã hội.

Phạm vi nghiên cứu:

- Về nền tảng: Tập trung phát triển trên nền tảng Android, có khả năng port sang iOS với chỉnh sửa tối thiểu nhờ đặc tính cross-platform của Unity.

- Về tính năng: Tập trung vào cốt lõi gameplay (board – tray – match-3 – booster – tutorial), hệ thống save/load local (JSON), hệ thống tim (heart) hồi phục theo thời gian, đồng bộ dữ liệu lên cloud, leaderboard, hệ thống bạn bè và gửi quà.
- Về thẩm mỹ: Sử dụng nguồn art có sẵn (asset 3D đơn giản, hiệu ứng DOTween), tập trung vào logic và kiến trúc thay vì phần đồ họa cao cấp.
- Về thời gian: Đồ án được thực hiện trong khoảng 4 tháng, từ giai đoạn phân tích đến hoàn thiện sản phẩm và viết báo cáo.

4. Phương pháp nghiên cứu

Trong quá trình thực hiện đồ án, em đã kết hợp nhiều phương pháp nghiên cứu khác nhau để đảm bảo chất lượng sản phẩm:

- Phương pháp nghiên cứu tài liệu: tham khảo tài liệu chính thức (Unity Documentation, Firebase Documentation), các sách chuyên ngành về phát triển game (Game Programming Patterns, Unity in Action), các khoá học trực tuyến và bài viết kỹ thuật.
- Phương pháp phân tích, thiết kế hệ thống: ứng dụng kiến thức về phân tích thiết kế hệ thống thông tin để mô hình hóa game ở cả góc nhìn cấu trúc (BFD, DFD, ERD) và hướng đối tượng (UML – Use Case, Sequence, Class diagram).
- Phương pháp thực nghiệm: cài đặt, kiểm thử và đo đặc hiệu năng trên các thiết bị thật và giả lập để đánh giá khả năng đáp ứng của hệ thống.
- Phương pháp khảo sát người dùng: tổ chức thử nghiệm sản phẩm với một nhóm người chơi mẫu nhằm thu thập phản hồi, từ đó cải thiện cảm giác chơi (game feel) và độ khó của các level.
- Phương pháp Agile/Scrum (rút gọn): chia công việc thành các sprint ngắn (1-2 tuần), liên tục review – cải tiến để bảo đảm tiến độ và chất lượng.

5. Cấu trúc của đồ án

Nội dung của báo cáo đồ án được tổ chức thành ba chương chính, ngoài phần Mở đầu và Kết luận:

Chương 1 – Tổng quan về bài toán và công nghệ, công cụ:

Trình bày hiện trạng thị trường game match-3 trên di động, đặc điểm và yêu cầu cụ thể của bài toán BlockJam3D, đồng thời giới thiệu chi tiết các công nghệ – công cụ được sử dụng trong quá trình phát triển: Unity Engine, ngôn ngữ C#, hệ thống Addressables, thư viện DOTween, nền tảng Firebase, hệ thống quản lý mã nguồn Git và quy trình phát triển phần mềm Agile được áp dụng.

Chương 2 – Phân tích thiết kế hệ thống:

Phân tích yêu cầu chức năng và phi chức năng của hệ thống, mô hình hóa theo cả hai hướng tiếp cận: hướng cấu trúc (BFD, DFD, ERD) và hướng đối tượng (Use Case, Sequence, Class diagram). Mỗi mô hình được kèm theo các bảng mô tả chi tiết để làm rõ chức năng và mối quan hệ giữa các thành phần.

Chương 3 – Xây dựng ứng dụng:

Trình bày chi tiết kiến trúc hệ thống, các mẫu thiết kế đã áp dụng, quá trình triển khai mã nguồn, công cụ Level Editor cho người quản trị, giao diện và chức năng dành cho người chơi, kết quả kiểm thử sản phẩm và các bước triển khai/đóng gói ứng dụng.

Phần Kết luận tổng hợp các kiến thức, kỹ năng đã rút ra được, đánh giá ưu nhược điểm của sản phẩm và đề xuất hướng phát triển trong tương lai. Cuối báo cáo là Danh mục tài liệu tham khảo được sử dụng trong quá trình thực hiện đồ án.

CHƯƠNG 1. TỔNG QUAN VỀ BÀI TOÁN VÀ CÔNG NGHỆ, CÔNG CỤ

1.1. Hiện trạng

Trong thập niên vừa qua, ngành công nghiệp game di động đã trở thành một trong những phân khúc tăng trưởng nhanh nhất của nền kinh tế giải trí số. Theo báo cáo Global Games Market Report của Newzoo, doanh thu game di động toàn cầu năm 2024 ước đạt 92,5 tỷ USD, chiếm khoảng 49% tổng doanh thu của toàn ngành game (187 tỷ USD). Tốc độ tăng trưởng kép hàng năm (CAGR) duy trì ở mức 6-8% trong giai đoạn 2020-2024 và được dự báo tiếp tục ổn định đến năm 2027.

Trong bức tranh đó, thể loại casual puzzle – đặc biệt là dòng match-3 / sort puzzle – luôn nằm trong top những thể loại thành công nhất về cả doanh thu lẫn lượt tải. Các tựa game tiêu biểu trên thị trường có thể kể đến:

- Candy Crush Saga (King) – tựa game biểu tượng của thể loại, ra mắt từ năm 2012, đã đạt hơn 3 tỷ lượt tải trên toàn cầu và vẫn duy trì doanh thu hàng trăm triệu USD mỗi năm.
- Royal Match (Dream Games) – ra mắt năm 2021, hiện là một trong những game match-3 có doanh thu cao nhất, áp dụng meta game decorate ấn tượng.
- Block Blast (Hungry Studio) – game block puzzle 3D đơn giản nhưng cuốn hút, đạt top 1 free game trên App Store nhiều thị trường năm 2024.
- Toon Blast (Peak Games) – tựa game match-2 nổi tiếng, được Zynga mua lại với giá 1,8 tỷ USD năm 2020.
- Gardenscapes, Homescapes (Playrix) – kết hợp match-3 với meta game trang trí, tạo nên xu hướng "narrative match-3".

Hình 1.1. Thị phần các game thể loại Match Puzzle trên thị trường mobile (minh họa)

Tại Việt Nam, mặc dù có nhiều studio đã thử sức với thể loại này nhưng số lượng sản phẩm thành công ở quy mô quốc tế còn khá khiêm tốn. Đây là một thị trường ngách còn nhiều dư địa, đặc biệt với các sản phẩm có tính sáng tạo trong cơ chế gameplay (mechanics) và meta game (hệ thống tiến trình, sự kiện, kết bạn...). Nhu cầu giải trí của

người chơi Việt Nam cũng đang tăng cao cả về thời lượng chơi lẫn sự sẵn sàng chi tiêu cho game, theo báo cáo của Vietnam Game Industry 2024 (VIRESA).

Tuy nhiên, việc tự phát triển một game match-3 hoàn chỉnh – chứ không chỉ là clone đơn giản – đặt ra nhiều thách thức cho lập trình viên trẻ:

- Cơ chế gameplay tưởng đơn giản nhưng đòi hỏi nhiều thuật toán phức tạp: pathfinding, match detection, rearrangement animation, level balancing.
- Đòi hỏi nhiều mảng kiến thức: lập trình hướng đối tượng, design pattern, kiến trúc phần mềm, cấu trúc dữ liệu – giải thuật, tối ưu hiệu năng, kết nối cloud, UI/UX, animation.
- Hiệu năng phải đạt mức cao trên các thiết bị cấp thấp – trung, do người dùng casual thường không dùng thiết bị flagship.
- Cần các công cụ hỗ trợ thiết kế nội dung (Level Editor) để mở rộng game về sau mà không phụ thuộc vào lập trình viên.

Hình 1.2. Giao diện một số game match-3 nổi tiếng (Candy Crush, Block Blast, Toon Blast)

Trên cơ sở phân tích hiện trạng nêu trên, đề tài đồ án "Thiết kế và xây dựng trò chơi giải đố BlockJam3D trên nền tảng Mobile" được lựa chọn với mong muốn xây dựng một sản phẩm game match-3 hoàn chỉnh, có kiến trúc rõ ràng, hiệu năng tốt và có khả năng mở rộng trong tương lai. Sản phẩm cũng đóng vai trò là một bài tập tổng hợp giúp người làm đồ án hệ thống lại toàn bộ kiến thức trong nhiều môn học: lập trình hướng đối tượng, phân tích thiết kế hệ thống, cấu trúc dữ liệu và giải thuật, cơ sở dữ liệu, mạng máy tính, công nghệ phần mềm.

1.2. Bài toán

Trên cơ sở phân tích hiện trạng đã trình bày ở mục 1.1, đề tài hướng đến xây dựng một game di động thuộc thể loại match-3 / sort puzzle với các đặc trưng sau:

1.2.1. Mô tả gameplay cốt lõi

Trò chơi BlockJam3D là một game giải đố 3D, người chơi cần thực hiện các thao tác để dọn sạch bàn cờ. Cụ thể:

- Bàn chơi (Board) là một lưới 2D chứa các khối (block) thuộc 7 màu sắc khác nhau (xanh dương, nâu, xanh lá, hồng, đỏ, tím, vàng), bên cạnh đó có các vật thể chặn như tường (Wall), thùng chứa (Container), barrel và các máng sinh khối (GridSpot).
- Bên dưới bàn chơi là một khay (CellPlay) chứa tối đa 7 ô. Người chơi nhấn vào một khối trên bàn → khối đó sẽ tự động di chuyển theo đường BFS xuống khay nếu có đường thông.
- Khi trong khay có ít nhất 3 khối cùng loại liên kề nhau, hệ thống sẽ tự kích hoạt Match-3: ba khối đó sẽ được hợp nhất, phát hiệu ứng và biến mất.
- Mỗi level gồm 3 round (board_1, board_2, board_3). Hoàn thành cả 3 round = Thắng level (Win). Nếu khay đầy 7 ô mà không match được = Thua (Lose) và mất 1 tim.
- Người chơi có 4 loại booster để hỗ trợ: Undo (hoàn tác), Add (thêm 3 ô khay), Shuffle (xáo trộn bàn cờ), Magnet (hút các khối cùng loại).
- Hệ thống tim (heart) giới hạn lượt chơi, tối đa 5 tim, hồi 1 tim mỗi 10 phút.
- Hệ thống xã hội: leaderboard toàn cầu, kết bạn, gửi quà booster / heart cho bạn bè.

1.2.2. Yêu cầu của người sử dụng

Phân tích các đối tượng người dùng và yêu cầu của họ:

a) Đối với người chơi (Player – Khách):

- Giao diện thân thiện, đẹp, dễ hiểu cho người dùng phổ thông (không cần đọc nhiều hướng dẫn).
- Cơ chế gameplay đơn giản, dễ chơi nhưng khó thắng, có chiều sâu chiến thuật.
- Hệ thống tutorial dẫn dắt người chơi qua các thao tác cơ bản ở level 1-2.
- Có cảm giác phản hồi tốt (game feel): hiệu ứng animation mượt, âm thanh, rung haptic khi tương tác.
- Hệ thống tiến trình rõ ràng (level, coin, booster) để duy trì động lực chơi tiếp.
- Có khả năng chơi offline (toàn bộ gameplay không yêu cầu internet) và tự đồng bộ khi online.

- Có chức năng đăng nhập Google để đồng bộ dữ liệu giữa các thiết bị.
- Có thể kết bạn, theo dõi thứ hạng, gửi và nhận quà booster/heart từ bạn bè.

b) Đối với người quản trị nội dung (Admin – Level Designer):

- Có công cụ giúp tạo và chỉnh sửa level mới mà không cần can thiệp source code.
- Có thể thiết kế lưới ô trực quan (toolbar palette) và lưu lại dưới dạng asset trong dự án.
- Hệ thống dễ mở rộng để thêm loại block mới, loại booster mới.
- Có công cụ giám sát người dùng (xem qua Firebase Console): danh sách user, level đạt được, lịch sử gửi quà.

c) Đối với nhà phát triển (Developer):

- Kiến trúc rõ ràng, có separation of concerns (mỗi module một nhiệm vụ).
- Sử dụng các design pattern phù hợp (Singleton, Object Pool, Event Bus, State Machine).
- Có quản lý mã nguồn (Git) chuyên nghiệp.
- Có tài liệu thiết kế và hướng dẫn vận hành đầy đủ.

1.2.3. Phạm vi giới hạn

Trong khuôn khổ đồ án, các giới hạn sau được đặt ra:

- Số lượng level ban đầu: 10 level x 3 round = 30 board.
- Số loại block: 7 màu cơ bản, có thêm trạng thái bị barrel chặn.
- Số loại booster: 4 (Undo, Add, Shuffle, Magnet).
- Hệ thống tim: max 5, regen 10 phút/tim, giá mua 500 coin/tim.
- Phần thưởng win: 100 coin/round.
- Giới hạn gửi quà: tối đa 3 lần/ngày (theo timezone UTC+7), bao gồm cả heart và booster.
- Ngôn ngữ hỗ trợ: Tiếng Việt và Tiếng Anh (qua hệ thống Localization tự xây).
- Chưa tích hợp In-App Purchase (IAP) và quảng cáo (Ads) – để dành cho phase phát hành thực tế.

1.3. Công nghệ, công cụ

Để xây dựng được sản phẩm BlockJam3D đáp ứng đầy đủ các yêu cầu nêu trên, đề tài đã lựa chọn một bộ công nghệ và công cụ phù hợp, bao gồm: Unity Engine làm nền tảng phát triển, C# làm ngôn ngữ lập trình, Visual Studio Code/Rider làm IDE, hệ thống Addressables để quản lý tài nguyên, thư viện DOTween cho hiệu ứng animation, Firebase Firestore và Realtime Database làm backend cloud, Git làm hệ thống quản lý mã nguồn, và áp dụng quy trình phát triển Agile/Scrum rút gọn.

1.3.1. Unity Engine

Unity là một game engine đa nền tảng do Unity Technologies phát triển. Ra đời năm 2005, đến nay Unity đã trở thành một trong những engine phổ biến nhất thế giới, được sử dụng bởi hơn 60% các nhà phát triển game indie và nhiều studio lớn. Phiên bản được sử dụng trong đề tài là Unity 6000.0.60f1 với Universal Render Pipeline (URP) – pipeline rendering tối ưu cho các thiết bị di động.

Các lý do chính khiến Unity được lựa chọn:

- Cross-platform: hỗ trợ build cho hơn 25 nền tảng (Android, iOS, Windows, macOS, WebGL...) từ cùng một codebase.
- Ngôn ngữ lập trình C# – ngôn ngữ hướng đối tượng mạnh, có nhiều tài liệu và cộng đồng lớn.
- Asset Store với hàng trăm nghìn asset và plugin miễn phí/trả phí.
- Hệ thống Component-Entity-System (CES) cho phép tổ chức code linh hoạt theo composition over inheritance.
- Tích hợp sẵn nhiều subsystem mạnh mẽ: Physics (PhysX), Audio, Animation, UI (UGUI/UI Toolkit), Networking, Profiler, Input System, Addressables.
- Khả năng tạo Editor extension giúp xây dựng các công cụ tùy biến (như Level Editor) bằng C# IMGUI hoặc UI Toolkit.

Hình 1.3. Kiến trúc tổng thể Unity Engine

Hình 1.4. Giao diện Unity Editor 6000.0.60f1

So sánh Unity với một số engine khác:

Tiêu chí	Unity 6	Unreal Engine 5	Godot 4	Cocos Creator 3
Ngôn ngữ chính	C#	C++ / Blueprint	GScript / C#	TypeScript / JavaScript
Hiệu năng đồ họa	Tốt (URP/HDRP)	Tốt nhất (Nanite, Lumen)	Trung bình	Trung bình
Phù hợp game 2D/Mobile	Rất tốt	Trung bình	Tốt	Tốt
Asset Store	Rất lớn	Lớn	Nhỏ	Trung bình
Học cong	Trung bình	Cao	Thấp	Trung bình
License	Miễn phí < 100K USD	Royalty 5% > 1M USD	MIT - hoàn toàn miễn phí	Miễn phí

Bảng 1.1. So sánh các game engine phổ biến cho phát triển game mobile

Có thể thấy Unity là lựa chọn cân bằng giữa hiệu năng đồ họa, độ phổ biến (tài liệu, plugin nhiều), khả năng phát triển game 2D/mobile và độ dốc học tập vừa phải – rất phù hợp với một đề tài đồ án tốt nghiệp.

Một số khái niệm cơ bản trong Unity được sử dụng trong dự án:

- **GameObject**: đối tượng cơ bản trong Scene, là container chứa các Component.
- **Component**: thành phần gán vào GameObject để thêm hành vi (Transform, Renderer, MonoBehaviour script...).
- **Scene**: cảnh – tập hợp các GameObject hợp thành một màn hình của game. Dự án có 5 scene chính.
- **Prefab**: GameObject template lưu dưới dạng asset, có thể được Instantiate nhiều lần.
- **ScriptableObject (SO)**: asset chứa dữ liệu, tách rời khỏi GameObject, dùng cho cấu hình level, booster...
- **MonoBehaviour**: base class cho mọi script gán vào GameObject; cung cấp lifecycle như Awake, Start, Update, OnEnable, OnDisable, OnDestroy.

1.3.2. Ngôn ngữ lập trình C#

C# (đọc là C-Sharp) là ngôn ngữ lập trình do Microsoft phát triển, được sử dụng làm ngôn ngữ scripting chính của Unity. C# là ngôn ngữ hướng đối tượng mạnh mẽ, cú pháp tương đồng với Java và C++, có nhiều tính năng hiện đại như async/await, LINQ, generics, pattern matching.

Phiên bản C# được Unity 6 hỗ trợ là C# 9 (với một số tính năng C# 10 thử nghiệm). Trong dự án BlockJam3D, các tính năng C# được sử dụng nhiều bao gồm:

- Hệ thống Generics: dùng cho các Singleton<T>, các Manager pattern.
- Coroutine (IEnumerator): dùng cho các hành động bất đồng bộ trong Unity (load scene, animation chuỗi).
- Delegate / Event / Action<T>: dùng cho hệ thống Event Bus.
- Async/Await + Task: dùng cho việc load Addressables, gọi Firestore/RTDB.
- LINQ: dùng để xử lý collection (lọc, sắp xếp danh sách block, level).
- Nullable reference types và pattern matching: làm code an toàn và rõ ràng hơn.

1.3.3. IDE và công cụ phát triển

Các IDE và công cụ hỗ trợ được sử dụng trong dự án:

- Visual Studio Code (cho lập trình C# với extension Unity và .NET Tools) – nhẹ, mở nhanh, miễn phí.
- JetBrains Rider (IDE chuyên nghiệp cho C# và Unity) – có khả năng refactor mạnh, gợi ý code thông minh.
- Unity Editor 6000.0.60f1 – môi trường chính để chỉnh sửa scene, prefab, asset.
- Unity Hub – công cụ quản lý phiên bản Unity và project.
- Git + GitHub – quản lý mã nguồn và lưu trữ trực tuyến.
- Plastic SCM (đi kèm Unity) – tùy chọn cho version control đặc thù cho asset binary lớn.

1.3.4. Hệ thống Addressables

Addressables là hệ thống quản lý tài nguyên (asset) thế hệ mới của Unity, ra đời để thay thế Resources.Load và AssetBundle cũ. Addressables cho phép:

- Load asset theo address (chuỗi tên gọi) hoặc label (gán nhãn nhiều asset cùng nhóm).
- Load async (bất đồng bộ) tránh giật khi load nhiều asset.
- Tách asset thành các content pack riêng biệt, có thể tải về sau (Remote Asset).
- Quản lý memory tốt: tự release sau khi không còn handle nào reference đến asset.

Trong BlockJam3D, Addressables được sử dụng để load các nhóm prefab gameplay (Wall, Container, Block, GridSpot) và các ScriptableObject level (label "Level"). Một singleton AddressableManager được xây dựng để preload toàn bộ asset gameplay lúc khởi động, sau đó cache lại trong Dictionary để các module khác có thể tra cứu bằng tên một cách đồng bộ. Lý do dùng cache: tránh chi phí gọi Addressables.LoadAssetAsync lặp lại nhiều lần trong gameplay.

1.3.5. Thư viện DOTween

DOTween là thư viện tween animation cực kỳ phổ biến trong cộng đồng Unity, được phát triển bởi Demigiant. DOTween cung cấp API ngắn gọn để tạo animation cho mọi loại property: Transform position/rotation/scale, Color, Float, custom property...

Ưu điểm của DOTween:

- API ngắn gọn (chain method): transform.DOMove(target, 0.5f).SetEase(Ease.OutQuad).
- Hiệu năng cao: không tạo nhiều garbage collection.
- Phong phú: hỗ trợ Sequence, Yoyo, OnComplete callback, Punch animation.
- Tích hợp với UI: DOFade, DOPunchScale... cho text, image.

Trong dự án, DOTween được dùng khắp nơi: animation di chuyển block từ board xuống khay, animation match-3 (raise – merge – pop), animation booster Magnet, animation coin fly khi win, animation loop yoyo cho icon booster...

1.3.6. Firebase (Firestore + Realtime Database)

Firebase là nền tảng Backend-as-a-Service (BaaS) của Google, cung cấp nhiều dịch vụ cloud cho ứng dụng di động và web. Trong dự án này, hai dịch vụ chính được sử dụng:

- **Cloud Firestore:**

CSDL NoSQL document-oriented, lưu dữ liệu dạng cây tài liệu (document) trong các collection. Hỗ trợ truy vấn phức tạp (where, orderBy, limit), giao dịch (transaction), batch write, listener realtime. Trong dự án, Firestore là source of truth cho dữ liệu người dùng (UserData/{uid}) bao gồm coin, level, hearts, boosters, danh sách bạn bè, các yêu cầu kết bạn đang chờ và đối với counter ID người dùng (ServerConfigs/UserCounter).

- **Realtime Database (RTDB):**

CSDL cây JSON realtime, đẩy thay đổi ngay lập tức đến client thông qua WebSocket. Trong dự án, RTDB đóng vai trò là kênh thông báo (notification bus): khi A gửi yêu cầu kết bạn cho B, một node ngắn được tạo trong RTDB, B đang lắng nghe sẽ nhận event ChildAdded, hiển thị thông báo, sau đó tự xóa node để node hoạt động như inbox một lần thay vì log dài. Tương tự, các event chấp nhận/từ chối kết bạn, gửi quà cũng đi qua RTDB.

Việc kết hợp Firestore + RTDB cho phép tận dụng ưu điểm của cả hai: Firestore có khả năng truy vấn mạnh, lưu trữ bền vững, trong khi RTDB có độ trễ rất thấp – phù hợp cho các thông báo realtime mà không cần truy vấn lại sau đó. Cả hai dịch vụ đều có gói Spark (miễn phí) đủ dùng cho mục đích thử nghiệm và phát triển.

1.3.7. Git và quy trình quản lý mã nguồn

Git là hệ thống quản lý mã nguồn phân tán (distributed version control) phổ biến nhất hiện nay. Trong dự án, Git được sử dụng kết hợp với GitHub để lưu trữ remote và làm việc nhóm.

Một số quy ước được áp dụng:

- Branching: main là branch chính, các tính năng được phát triển trong feature branch (feat/leaderboard, feat/booster-magnet...), sau đó merge về main qua Pull Request.
- Commit message: theo Conventional Commits (feat:, fix:, refactor:, docs:, chore:...).

- .gitignore: bỏ qua thư mục Library/, Temp/, Logs/, UserSettings/, .csproj sinh tự động.
- Git LFS: dùng cho các file asset binary lớn (ảnh PNG cao độ phân giải, audio...).

1.4. Quy trình phát triển phần mềm

Dự án được tổ chức theo mô hình Agile/Scrum rút gọn, vì đặc thù đồ án thường có một người chính làm developer và thời gian hạn chế (~4 tháng). Cụ thể, quy trình thực hiện bao gồm các giai đoạn sau:

1.4.1. Khảo sát yêu cầu và lập kế hoạch (1 tuần)

Trong tuần đầu tiên, em đã tiến hành khảo sát các tựa game tham chiếu (Block Blast, Toon Blast), liệt kê các tính năng tối thiểu (MVP) và các tính năng mở rộng. Một bản backlog được lập, sau đó các task được phân chia theo độ ưu tiên (Must have, Should have, Could have) và sắp xếp vào các sprint.

1.4.2. Phân tích, thiết kế (2 tuần)

Giai đoạn này tập trung vào việc phân tích yêu cầu chức năng/phi chức năng, thiết kế các sơ đồ BFD, DFD, ERD, Use Case, Sequence, Class. Cũng trong giai đoạn này em đã thiết kế kiến trúc tổng thể (Singleton + Event Bus + State Machine), xác định các module chính và mối quan hệ giữa chúng (xem chi tiết ở chương 2 và 3).

1.4.3. Cài đặt – kiểm thử (9 tuần, chia theo sprint)

Giai đoạn cài đặt được chia thành các sprint ~2 tuần. Mỗi sprint kết thúc bằng một bản demo có thể chạy được trên Editor và đầu sprint sau là buổi review – điều chỉnh backlog. Bảng sprint cụ thể:

Sprint	Mục tiêu chính	Output
Sprint 1 (tuần 4-5)	Bootstrap, GameManager, Scene flow, UIManager	Init → Menu → Gameplay scaffold
Sprint 2 (tuần 6-7)	BoardCtrl, LevelData, AddressableManager, basic gameplay	Có thể load level, click khối di chuyển
Sprint 3 (tuần 8-9)	CellPlayCtrl, Match-3, Win/Lose, Object Pool	Gameplay loop hoàn chỉnh
Sprint 4 (tuần 10-11)	4 Booster, Tutorial, Save/Load	Game có đầy đủ tính năng

Sprint	Mục tiêu chính	Output
11)		offline
Sprint 5 (tuần 12-13)	Firebase Firestore + RTDB, Auth, Leaderboard	Online features
Sprint 6 (tuần 14-15)	Friend system, Send Gift, Localization, Polish	Production-ready

Bảng 3.5. Bảng các module triển khai theo tuần**1.4.4. Kiểm thử và đóng gói (3 tuần)**

Sau khi tính năng đã ổn định, em tiến hành kiểm thử trên Editor và trên thiết bị Android thật (Samsung A12, Xiaomi Redmi Note 10, Realme C25Y). Các loại kiểm thử bao gồm:

- Functional test: kiểm tra từng tính năng theo bảng kịch bản (Test Case).
- Performance test: đo FPS, GC alloc bằng Unity Profiler.
- Compatibility test: test trên các độ phân giải khác nhau (16:9, 19.5:9, tablet 4:3).
- User test: cho 5-10 người chơi thử và thu phản hồi.

1.4.5. Viết báo cáo (1-2 tuần)

Báo cáo được viết song song với quá trình phát triển – mỗi sprint kết thúc, các tài liệu thiết kế và mô hình được cập nhật ngay. Tuần cuối tập trung tổng hợp, biên tập, và làm slide thuyết trình.

Hình 1.5. Quy trình phát triển phần mềm Agile/Scrum

Việc áp dụng Agile rút gọn mang lại nhiều lợi ích thực tế: khả năng phản hồi nhanh khi gặp vấn đề kỹ thuật phát sinh, không bị "đóng băng" thiết kế ngay từ đầu, các tính năng được kiểm thử sớm – tránh tình huống đến phút cuối mới phát hiện lỗi nghiêm trọng. Tuy nhiên, người làm đồ án cần kỷ luật trong việc theo dõi tiến độ – em đã sử dụng Trello để quản lý backlog và Notion để lưu trữ tài liệu thiết kế trong quá trình phát triển.

1.5. Kết luận chương 1

Chương 1 đã trình bày tổng quan về bài toán và bối cảnh của đề tài: thị trường game match-3 toàn cầu vẫn đang tăng trưởng mạnh, là một mảnh đất tốt cho việc thử sức trong ngành game indie. Mục tiêu của đề tài là xây dựng một sản phẩm match-3 hoàn

chính với cốt lõi gameplay rõ ràng, các tính năng meta và xã hội phong phú, kiến trúc rõ ràng và hiệu năng tốt trên thiết bị tầm trung.

Chương cũng đã giới thiệu chi tiết các công nghệ – công cụ được sử dụng: Unity Engine 6000.0.60f1 (URP) làm nền tảng, C# làm ngôn ngữ, Visual Studio Code/Rider làm IDE, Addressables quản lý tài nguyên, DOTween cho animation, Firebase (Firestore + RTDB) cho backend cloud, Git cho version control. Mô hình phát triển Agile/Scrum rút gọn được áp dụng để quản lý tiến độ trong 4 tháng.

Các nội dung này là tiền đề cho phần phân tích – thiết kế hệ thống chi tiết được trình bày ở Chương 2.

CHƯƠNG 2. PHÂN TÍCH THIẾT KẾ HỆ THỐNG

2.1. Yêu cầu chức năng

Yêu cầu chức năng (Functional Requirements) là những việc hệ thống phải làm được, hay nói cách khác là các chức năng mà người dùng có thể thực hiện thông qua hệ thống. Đối với hệ thống BlockJam3D, các yêu cầu chức năng được phân nhóm theo hai vai trò chính: Player (người chơi) và Admin/Level Designer (người làm nội dung). Các nhà phát triển (Developer) cũng có một số yêu cầu liên quan đến công cụ và quy trình, được trình bày ở phần sau.

2.1.1. Yêu cầu chức năng cho người chơi (Player)

ID	Tên chức năng	Mô tả
FR-P-01	Khởi động ứng dụng	Khi mở ứng dụng, hệ thống tự động load dữ liệu người chơi từ file local, hiển thị màn hình loading, sau đó chuyển tới menu chính.
FR-P-02	Xem màn hình chính	Menu chính hiển thị: nút PLAY (level hiện tại), coin hiện có, số tim, đồng hồ đếm ngược tim, nút settings, nút mở các tab Shop/Mission/League/Collection.
FR-P-03	Bắt đầu chơi level	Người chơi bấm PLAY → hệ thống load scene Gameplay, khởi tạo level theo UserData.level, hiển thị bàn cờ và khay.
FR-P-04	Tương tác với khối block	Người chơi tap vào một khối → khối di chuyển theo BFS đến khay nếu có đường đi thông; ngược lại hiện hiệu ứng phản hồi.
FR-P-05	Cơ chế Match-3	Khi khay chứa ≥ 3 khối liên kề cùng loại → tự động kích hoạt animation merge, các khối biến mất, các khối khác xếp lại.
FR-P-06	Sử dụng Booster Undo	Hoàn tác hành động cuối cùng (di chuyển khối hoặc match-3).
FR-P-07	Sử dụng Booster Add	Mở thêm 3 ô khay phụ phía dưới khay chính.
FR-P-08	Sử dụng Booster Shuffle	Xáo trộn vị trí các khối thường trên bàn, giữ nguyên Wall/Container/GSP/Barrel.
FR-P-09	Sử dụng Booster Magnet	Hút 3 khối cùng loại (kể cả từ trên bàn) về khay, kích hoạt match-3 ngay.

ID	Tên chức năng	Mô tả
FR-P-10	Win Game	Hoàn thành 3 round của level → +100 coin, level + 1, lưu dữ liệu.
FR-P-11	Lose Game	Khay đầy 7 ô không match được → mất 1 tim, có thể Try Again hoặc về Menu.
FR-P-12	Pause / Resume	Mở popup Settings trong gameplay, có thể bật/tắt sound, vibration, thoát level.
FR-P-13	Hệ thống Tim (Heart)	Tối đa 5 tim, regen 1 tim/10 phút (tính cả khi offline), có thể mua thêm 500 coin/tim, xem ad, xin từ bạn bè.
FR-P-14	Xem Settings	Bật/tắt âm thanh, rung, đăng nhập Google.
FR-P-15	Đăng nhập Google	Liên kết tài khoản Google qua Unity Player Accounts → đồng bộ với Firestore.
FR-P-16	Xem Leaderboard Players (Global)	Top 100 người chơi theo level, có thứ hạng riêng cho mình.
FR-P-17	Xem Leaderboard Friends	Danh sách bạn bè theo level, có nút Add Friend, Friend Request.
FR-P-18	Tìm và kết bạn theo ID	Nhập prefix ID (>=4 ký tự) để tìm user, gửi yêu cầu kết bạn.
FR-P-19	Chấp nhận / Từ chối kết bạn	Xem danh sách Friend Request, chấp nhận hoặc từ chối.
FR-P-20	Gửi quà cho bạn	Gửi booster (Undo/Add/Shuffle) hoặc Heart, giới hạn 3 lần/ngày.
FR-P-21	Nhận thông báo realtime	Nhận thông báo khi có yêu cầu kết bạn, được chấp nhận, hoặc được tặng quà.
FR-P-22	Tutorial level 1-2	Hướng dẫn cơ bản: tap block (lvl 1 round 0), order (lvl 1 round 1), pipe (lvl 2 round 1).
FR-P-23	Đổi ngôn ngữ	Chuyển giữa Tiếng Việt và Tiếng Anh.
FR-P-24	Lưu/load tiến trình offline	Mọi dữ liệu coin, level, booster, hearts được lưu vào file UserData.json trong persistentDataPath.

Bảng 2.1. Bảng yêu cầu chức năng cho người chơi (Player)

2.1.2. Yêu cầu chức năng cho người quản trị nội dung (Admin / Level Designer)

ID	Tên chức năng	Mô tả
FR-A-01	Mở Level Editor	Menu "Game Tools → Level Editor" trong Unity Editor mở cửa sổ Level Editor.
FR-A-02	Tạo level mới	Đặt width / height cho lưới, hệ thống tự tạo mảng prefabNames có kích thước tương ứng.
FR-A-03	Vẽ lưới ô bằng toolbar palette	Chọn loại tile (Empty / Wall / Container / 1..7 / 1B..7B / GSPDown / GSPUp / GSPLeft / GSPRight / GSPBottomRight) rồi click vào ô lưới.
FR-A-04	Cài đặt spawn count cho GSP	Khi đặt tile GSP, panel hiện ra để cấu hình số lượng khối GSP đó sẽ sinh.
FR-A-05	Lưu level thành asset	Lưu thành asset Level_X(board_N).asset trong thư mục Assets/LevelData/Level_X/.
FR-A-06	Mở level đã có để chỉnh sửa	Chọn asset đang có, các giá trị width/height/prefabNames/gsp được load vào editor.
FR-A-07	Xem qua Firebase Console	Quản trị xem danh sách user, level, friend request, lịch sử gửi quà qua Firebase Console (chức năng built-in của Firebase).
FR-A-08	Reset dữ liệu test	Trong Editor, có nút "Delete Save" trong UserData để xóa file save phục vụ test.

Bảng 2.2. Bảng yêu cầu chức năng cho người quản trị (Admin/Level Designer)

2.2. Yêu cầu phi chức năng

Yêu cầu phi chức năng (Non-Functional Requirements) đề cập đến các thuộc tính chất lượng của hệ thống – không phải chức năng cụ thể nhưng quyết định mức độ hài lòng của người dùng và khả năng vận hành lâu dài của sản phẩm.

ID	Loại	Yêu cầu cụ thể
NFR-01	Hiệu năng	Trên thiết bị Android tầm trung (RAM 4GB, Snapdragon 6xx hoặc tương đương), game phải đạt tối thiểu 60 FPS ổn định trong gameplay, không drop dưới 45 FPS quá 1 giây.
NFR-02	Hiệu năng	Thời gian khởi động (Splash → Main Menu) không quá 5 giây.
NFR-03	Hiệu năng	Thời gian chuyển scene GamePlay → load level không quá 2 giây.

ID	Loại	Yêu cầu cụ thể
NFR-04	Bộ nhớ	Sử dụng RAM ở mức < 600 MB trong gameplay; không bị memory leak khi chơi nhiều level liên tục.
NFR-05	Mượt mà animation	Animation match-3, di chuyển khối phải mượt, không bị giật khi nhiều khối di chuyển đồng thời.
NFR-06	Khả năng sử dụng offline	Toàn bộ gameplay (level, save, booster, heart) chạy hoàn toàn offline.
NFR-07	Đồng bộ	Khi có internet và đã đăng nhập Google, dữ liệu được đồng bộ với Firestore. Khi offline, các thay đổi được giữ ở local và push lên khi reconnect.
NFR-08	Tính bảo mật	Dữ liệu nhạy cảm (token Firebase, Google) không hard-code trong code. Sử dụng Firebase Security Rules để giới hạn quyền truy cập (chỉ user đó được sửa UserData của mình).
NFR-09	Tính khả dụng	UI thân thiện với người chơi phổ thông, tap-target đủ lớn ($\geq 48dp$), font chữ rõ ràng.
NFR-10	Khả năng mở rộng	Kiến trúc cho phép thêm booster mới, level mới, ngôn ngữ mới mà không phải sửa core code.
NFR-11	Khả năng bảo trì	Mã nguồn được tổ chức theo module rõ ràng, có comment, có tài liệu kiến trúc.
NFR-12	Đa nền tảng	Cùng codebase build được cho Android và iOS (chỉ cần đổi target trong Build Settings).
NFR-13	Tính ổn định	Tỷ lệ crash dưới 1% session (theo metric Firebase Crashlytics – nếu tích hợp).
NFR-14	Đa ngôn ngữ	Hỗ trợ Tiếng Việt và Tiếng Anh, có thể thêm ngôn ngữ khác.
NFR-15	Trải nghiệm cảm giác	Game feel tốt: có haptic feedback, SFX rõ ràng cho mọi hành động chính, hiệu ứng visual phong phú.

Bảng 2.3. Bảng yêu cầu phi chức năng

2.3. Mô hình hóa hệ thống – Hướng cấu trúc

Mô hình hóa hệ thống theo hướng cấu trúc tập trung vào việc phân rã hệ thống thành các chức năng nghiệp vụ và mô tả luồng dữ liệu giữa chúng. Ba sơ đồ chính được sử dụng là: Sơ đồ chức năng nghiệp vụ (BFD), Sơ đồ luồng dữ liệu (DFD) và Sơ đồ thực thể – liên kết (ERD).

2.3.1. Sơ đồ chức năng nghiệp vụ (BFD)

Sơ đồ BFD mô tả các nhóm chức năng chính của hệ thống và phân rã chúng thành các chức năng con. Đối với BlockJam3D, BFD tổng thể được phân thành 5 nhóm chức năng lớn:

Hệ thống BlockJam3D

- 1. Quản lý phiên (Session Management)
 - 1.1 Khởi động ứng dụng & load asset
 - 1.2 Load dữ liệu người chơi từ local
 - 1.3 Đăng nhập Google (tùy chọn)
 - 1.4 Đồng bộ dữ liệu với cloud
- 2. Quản lý Gameplay
 - 2.1 Quản lý level (load, chuyển round)
 - 2.2 Quản lý bàn chơi (Board)
 - 2.3 Quản lý khay (CellPlay)
 - 2.4 Cơ chế Match-3
 - 2.5 Cơ chế thắng / thua
 - 2.6 Tutorial
- 3. Quản lý Booster
 - 3.1 Booster Undo
 - 3.2 Booster Add
 - 3.3 Booster Shuffle
 - 3.4 Booster Magnet
- 4. Quản lý Người chơi & Tài sản
 - 4.1 Quản lý Tim (Heart)
 - 4.2 Quản lý Coin
 - 4.3 Quản lý Booster Inventory
 - 4.4 Mua sắm (Shop) - chưa cài IAP
 - 4.5 Cài đặt (Settings)
- 5. Quản lý Xã hội
 - 5.1 Leaderboard (Friends / Global)
 - 5.2 Tìm kiếm bạn bè theo ID
 - 5.3 Gửi / chấp nhận / từ chối yêu cầu kết bạn
 - 5.4 Gửi quà (Booster / Heart) cho bạn
 - 5.5 Nhận thông báo realtime

Hình 2.1. Sơ đồ chức năng nghiệp vụ (BFD) tổng thể

Mỗi nhánh trong BFD trên tương ứng với một hoặc nhiều module trong cài đặt thực tế (xem chương 3). Ví dụ, nhánh 2 (Quản lý Gameplay) chính là module

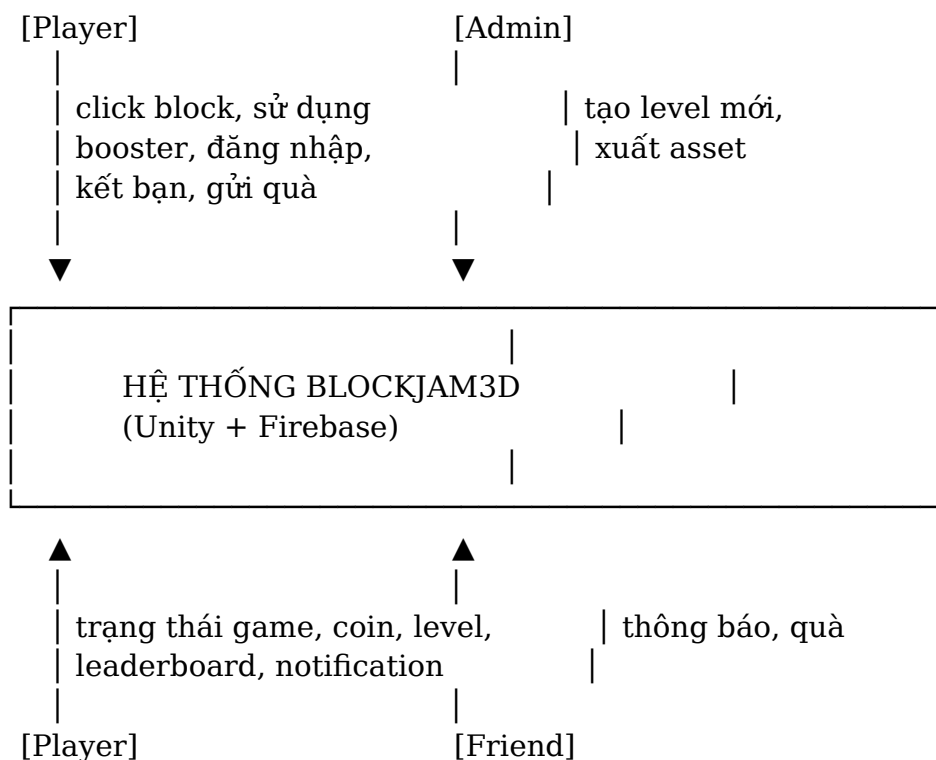
LevelManager và các sub-controller; nhánh 5 (Quản lý Xã hội) tương ứng với module UserDataFirebaseManager và LeaderBoardManager.

2.3.2. Sơ đồ luồng dữ liệu (DFD)

Sơ đồ DFD mô tả luồng dữ liệu di chuyển giữa các tiến trình và kho dữ liệu. Tại đây, DFD được vẽ ở hai mức: mức ngữ cảnh (level 0) và mức 1.

a) DFD mức 0 (Context Diagram):

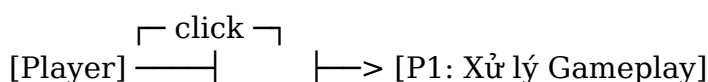
Mức này thể hiện hệ thống như một hộp đen, có 3 actor bên ngoài: Player (người chơi), Admin (người làm nội dung) và Friend (bạn của người chơi). Các luồng dữ liệu chính:

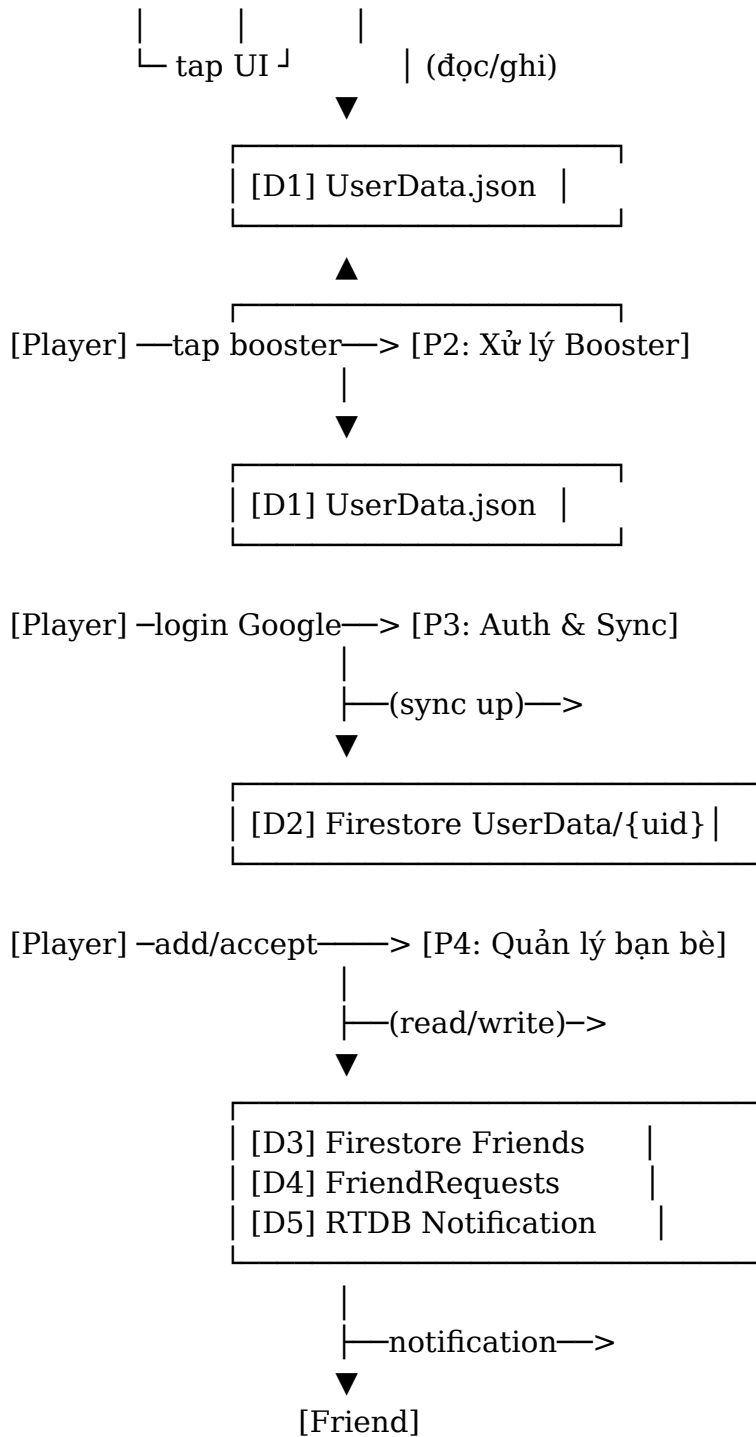


Hình 2.2. Sơ đồ luồng dữ liệu mức ngữ cảnh (DFD level 0)

b) DFD mức 1:

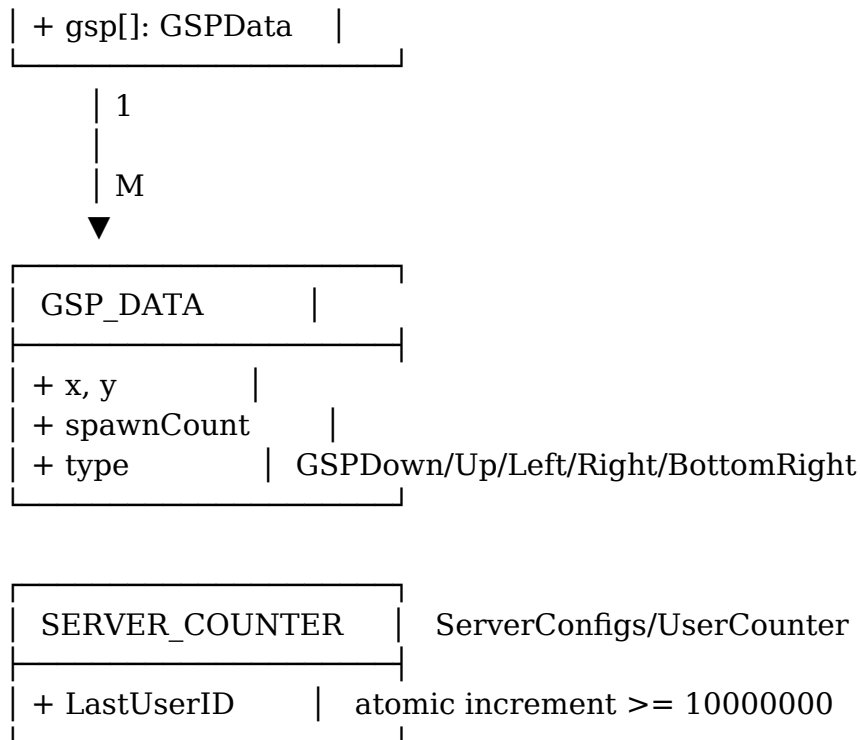
DFD mức 1 phân rã hộp đen thành các tiến trình chính: Xử lý gameplay, Xử lý booster, Quản lý người chơi, Quản lý mạng xã hội. Đồng thời, các kho dữ liệu chính cũng được thể hiện: UserData.json (local), Firestore UserData, Firestore Friends, RTDB Notification.





Hình 2.3. Sơ đồ luồng dữ liệu mức 1 (DFD level 1)

Sơ đồ này cho thấy rõ phần lớn dữ liệu được lưu cục bộ trong UserData.json (phục vụ chế độ offline), trong khi Firestore đóng vai trò là source of truth khi đăng nhập, và RTDB đóng vai trò channel thông báo realtime tới bạn bè.

**Hình 2.4. Sơ đồ thực thể – liên kết (ERD)**

Bảng mô tả chi tiết các thực thể:

Thuộc tính	Kiểu dữ liệu	Ràng buộc / mô tả
Id	string	Khoá chính, duy nhất, format "10000000"+ (>= 10000000)
Name	string	Tên hiển thị, mặc định Player + Id
Coin	int (>= 0)	Số coin hiện tại, mặc định 99999 (test) – sản phẩm thực sẽ là 0
Level	int (>= 1)	Level hiện tại của người chơi
Heart	int [0..5]	Số tim hiện tại
Frame	int (>= 0)	Frame avatar đã unlock (placeholder)
UnityPlayerId	string	ID Unity Player Account, dùng để link Google
LoginType	string	"Local" hoặc "Google"
CreatedAt	Timestamp	Server timestamp khi tạo user
LastLoginAt	Timestamp	Server timestamp lần đăng nhập gần nhất
LastSendBoosterDate	string (yyyy-MM-	Ngày cuối gửi quà (timezone UTC+7)

Thuộc tính	Kiểu dữ liệu	Ràng buộc / mô tả
	dd)	
SendBoosterCount	int [0..3]	Số lần đã gửi trong ngày, reset hàng ngày

Bảng 2.4. Bảng mô tả thực thể UserData

Thuộc tính	Kiểu dữ liệu	Ràng buộc / mô tả
Name	string	Khoá chính, định dạng Level_X(board_N)
width	int (>0)	Số cột của lưới board
height	int (>0)	Số hàng của lưới board
prefabNames	string[]	Mảng row-major chứa tên prefab cho từng ô, index = y*width + x. "" / null = ô trống.
alignment	bool	Cờ canh lề bàn chơi (offset camera)
totalUnits	int	Tổng số khối (dùng cho cân bằng spawn random)
gsp	GSPData[]	Danh sách mảng GridSpot trên board

Bảng 2.5. Bảng mô tả thực thể LevelData

Thuộc tính	Kiểu dữ liệu	Ràng buộc / mô tả
Id	string	Khoá chính (giống Id của user kia)
CreatedAt	Timestamp	Thời điểm trở thành bạn
FriendRequest.FromUserId	string (FK → User.Id)	ID người gửi yêu cầu
FriendRequest.ToUserId	string (FK → User.Id)	ID người nhận yêu cầu
FriendRequest.Status	enum	Pending / Accepted / Declined

Bảng 2.6. Bảng mô tả thực thể Friend và FriendRequest

2.4. Mô hình hóa hệ thống – Hướng đối tượng

Trái với hướng cấu trúc tập trung vào chức năng và luồng dữ liệu, hướng đối tượng (UML) tập trung vào các đối tượng (class) và tương tác giữa chúng. Ba sơ đồ chính được sử dụng: Use Case Diagram, Sequence Diagram và Class Diagram.

Mục	Nội dung
Tên Use Case	Chơi một level
Actor	Player
Mô tả	Người chơi vào màn hình gameplay và hoàn thành 3 round để vượt level.
Tiền điều kiện	App đang ở Menu, người chơi còn ≥ 1 tim.
Hậu điều kiện (thành công)	Level tăng 1, +100 coin, dữ liệu được lưu, popup Win hiện ra.
Hậu điều kiện (thất bại)	Mất 1 tim, popup Lose hiện ra.
Luồng chính	1. Player tap PLAY ở Main Menu. 2. Hệ thống load scene Gameplay, hiện ScreenGamePlay HUD. 3. LevelManager.Init() load LevelData round 0. 4. BoardCtrl spawn các block, wall, container, GSP. 5. Hệ thống chạy tutorial nếu level 1 hoặc 2. 6. Player tap khối block trên bàn → block di chuyển xuống khay (BFS). 7. Hệ thống tự kích hoạt Match-3 khi đủ 3 khối cùng loại liên kề. 8. Khi board hết và Round < 2 → NextRound; Round ≥ 2 → WinGame. 9. Lưu UserData, hiện popup Win.
Luồng phụ (Lose)	7'. Khay đầy 7 ô và không match được → checkLose true. 8'. Hệ thống gọi GameManager.LoseGame() → popup Lose, trừ 1 tim.
Tần suất	Rất cao (chức năng cốt lõi)

Bảng 2.7. Bảng mô tả Use Case "Chơi một level"

Mục	Nội dung
Tên Use Case	Sử dụng Booster
Actor	Player
Mô tả	Người chơi sử dụng 1 trong 4 booster để gỡ thế khó trên bàn.
Tiền điều kiện	Đang trong gameplay, BoosterCtrl.IsBusy == false, người chơi có booster (count > 0) hoặc đủ coin trả price.
Hậu điều kiện	Booster được kích hoạt, count - 1 (hoặc trừ coin nếu count = 0), trạng thái bàn cờ thay đổi tương ứng.

Mục	Nội dung
Luồng chính	1. Player tap nút booster trên HUD. 2. ScreenGamePlay đọc UserData.listBoosterCounters. 3. Nếu count > 0: count--, gọi BoosterXxx.Use(). Nếu count = 0 và coin >= price: coin -= price, gọi Use(). Ngược lại: phát NotifyContent "Không đủ coin". 4. BoosterCtrl.IsBusy = true (khóa input). 5. Logic riêng từng booster (xem chương 3). 6. BoosterCtrl.IsBusy = false. 7. SaveDataManager.Save().
Luồng phụ	Khi đang animation booster, các tap khác bị ignore.
Tần suất	Trung bình

Bảng 2.8. Bảng mô tả Use Case "Sử dụng Booster"

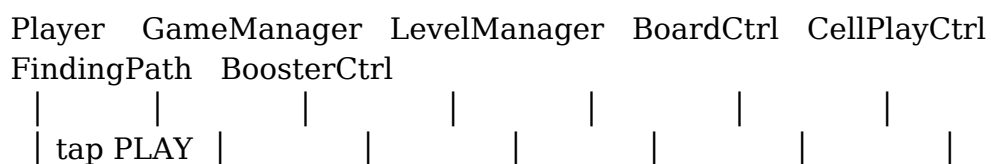
Mục	Nội dung
Tên Use Case	Đăng nhập Google
Actor	Player
Mô tả	Player liên kết tài khoản chơi với tài khoản Google qua Unity Player Account.
Tiền điều kiện	Có kết nối internet, Firebase đã initialized thành công, package com.unity.services.authentication có sẵn.
Hậu điều kiện (thành công)	UnityPlayerId, LoginType=Google, GoogleLinkedAt được set trong Firestore; nếu đã tồn tại trên cloud, dữ liệu (coin, level, hearts, boosters) được pull về local.
Luồng chính	1. Player mở PopupSettingsUIMain. 2. Tap nút "Sign in with Google". 3. UserDataFirebaseManager.LinkGoogleAccount() được gọi. 4. Unity Player Accounts mở dialog Google sign-in. 5. Sau khi user xác nhận → callback trả về unityPlayerId. 6. Firestore query WhereEqualTo("UnityPlayerId", id). 7. Nếu tìm thấy doc khác myUserId → pull dữ liệu về UserData, restart RTDB listener. 8. Nếu không → merge local doc với UnityPlayerId + LoginType + GoogleLinkedAt.
Luồng phụ (hủy)	User hủy dialog → không thay đổi gì, có thể thử lại.
Tần suất	Thấp – thường chỉ 1 lần

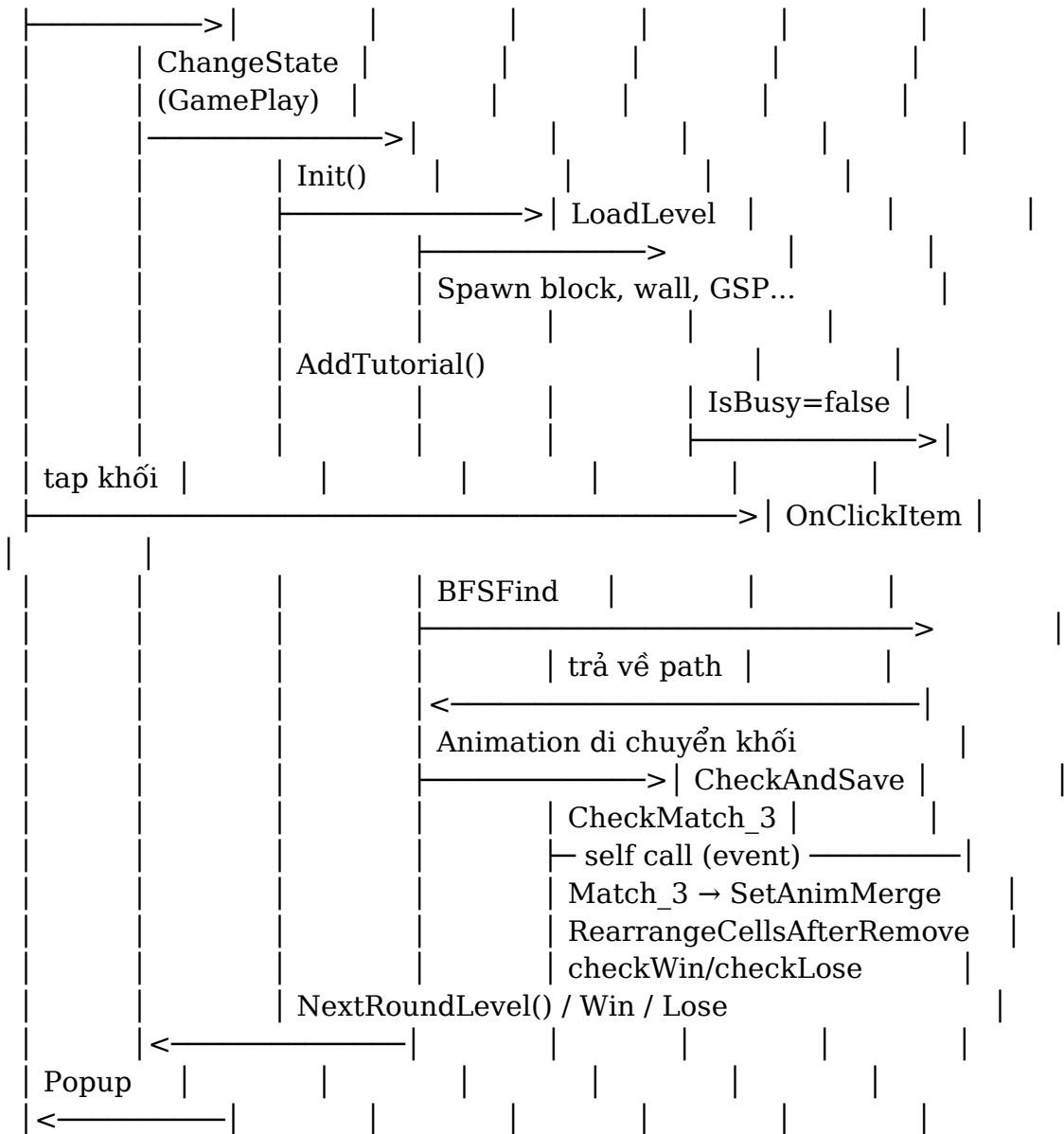
Bảng 2.9. Bảng mô tả Use Case "Đăng nhập Google"

Mục	Nội dung
Tên Use Case	Gửi quà cho bạn bè
Actor	Player (sender), Friend (receiver)
Mô tả	Player gửi 1 booster hoặc 1 heart cho 1 friend, giới hạn 3 lần/ngày.
Tiền điều kiện	Player đã đăng nhập Google, ToUser nằm trong danh sách Friends, sender có đủ booster/heart trong inventory, SendBoosterCount < 3 (timezone UTC+7).
Hậu điều kiện (thành công)	Sender count - 1, receiver count + 1, SendBoosterCount + 1, log gửi trên RTDB.
Luồng chính	1. Player mở PopupSendGift cho 1 friend. 2. Chọn loại quà (Undo/Add/Shuffle/Heart), tap Send. 3. SendBooster(from, to, name, amount) được gọi. 4. Firestore RunTransactionAsync: kiểm tra SendBoosterCount, debit sender, credit receiver, bump counter. 5. Transaction thành công → RTDB push send_booster/{toUid}/{<pushKey>. 6. Receiver lắng nghe ChildAdded → HeartManager.Add / UserData.listBoosterCounters update + SaveDataManager.Save. 7. Receiver RemoveValueAsync(node).
Luồng phụ	NOT_ENOUGH / LIMIT_REACHED / USER_NOT_FOUND → SendBoosterException → UIManager.NotifyContent("Lỗi tương ứng").
Tần suất	Trung bình (max 3 lần/ngày/user)

Bảng 2.10. Bảng mô tả Use Case "Gửi quà cho bạn bè"**2.4.2. Sơ đồ Sequence**

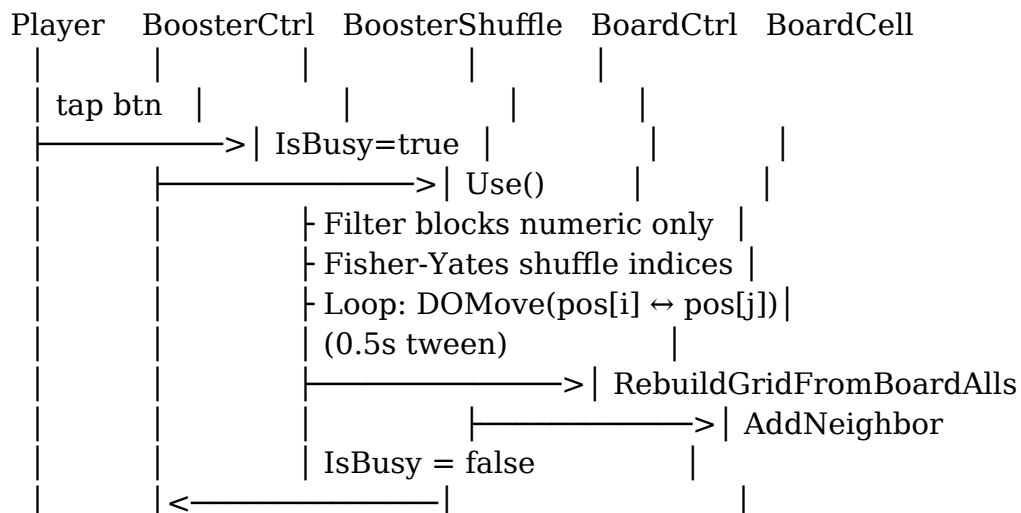
Sơ đồ Sequence mô tả thứ tự trao đổi thông điệp giữa các đối tượng theo thời gian khi thực hiện một use case cụ thể. Dưới đây là một số sơ đồ Sequence tiêu biểu.

a) Sequence – Chơi 1 round:



Hình 2.6. Sơ đồ Sequence – Chơi một round

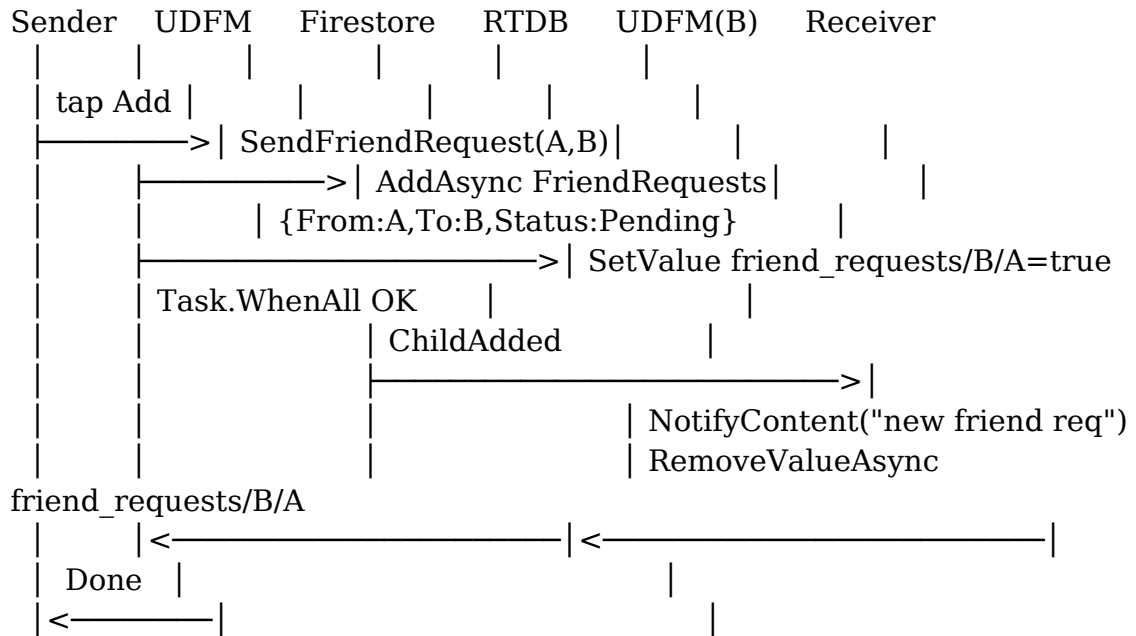
b) Sequence – Booster Shuffle:





Hình 2.7. Sơ đồ Sequence – Sử dụng booster Shuffle

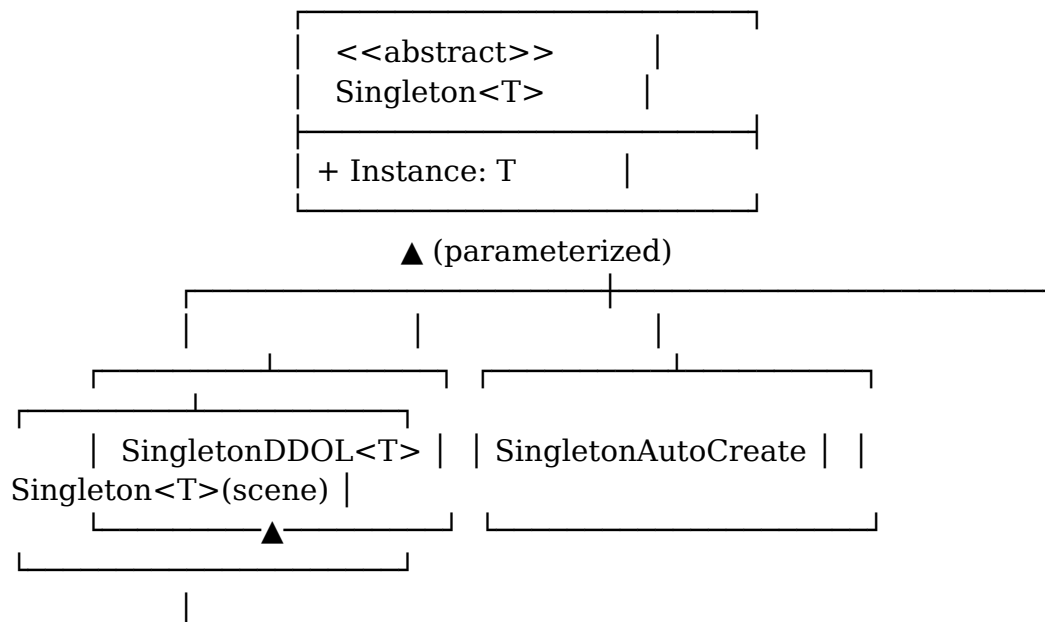
c) Sequence – Gửi yêu cầu kết bạn:

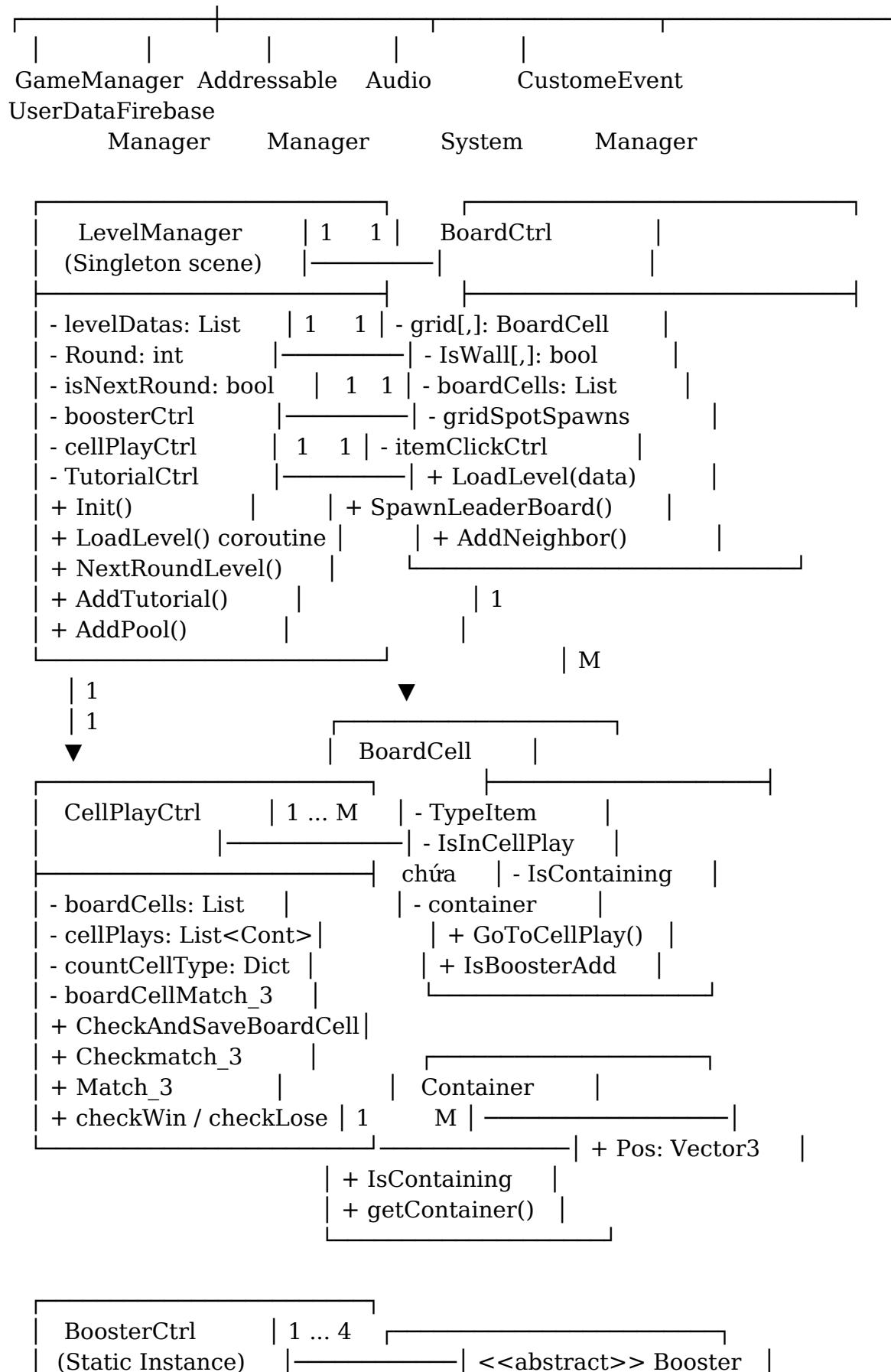


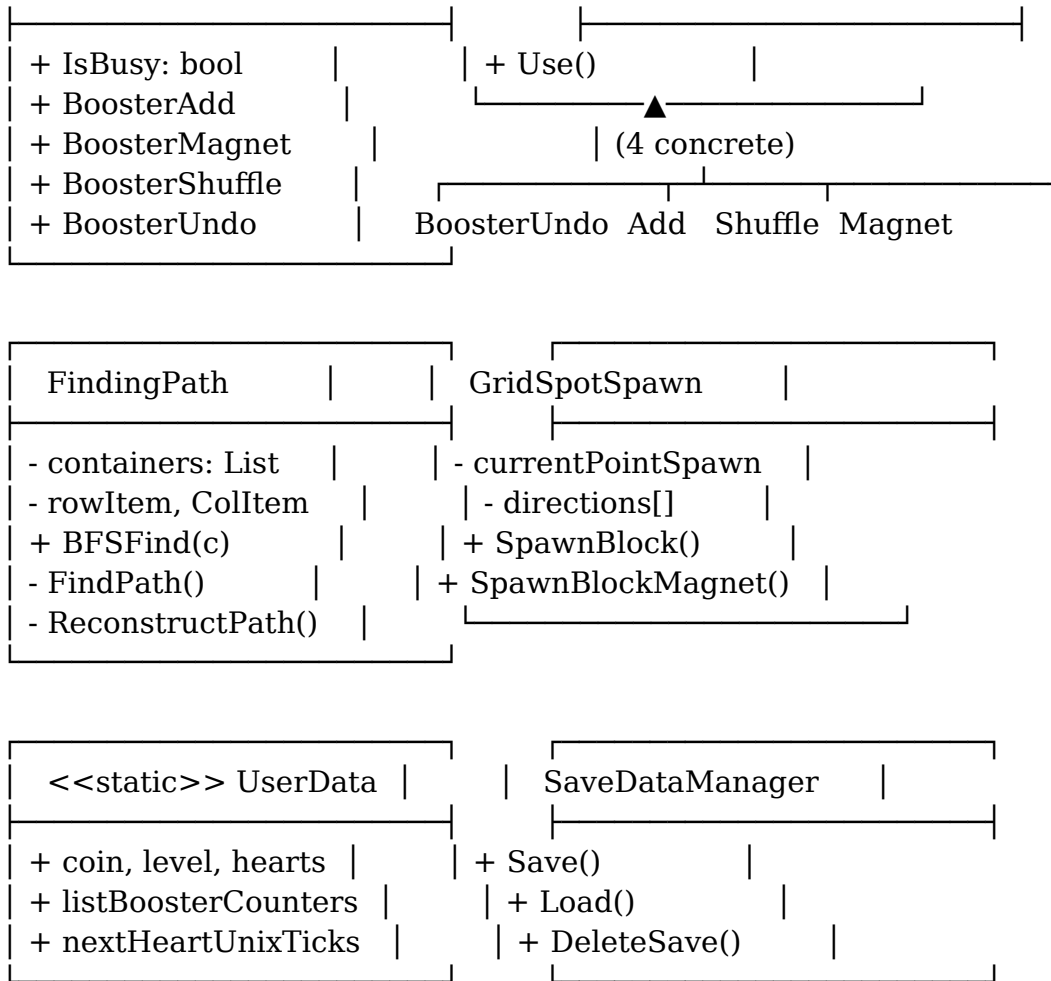
Hình 2.8. Sơ đồ Sequence – Gửi yêu cầu kết bạn

2.4.3. Sơ đồ Class

Sơ đồ Class mô tả các lớp (class) trong hệ thống và mối quan hệ giữa chúng (kế thừa, kết hợp, gộp, phụ thuộc). Do hệ thống có rất nhiều class, ở đây ta vẽ sơ đồ class tổng thể chỉ thể hiện các class quan trọng nhất.

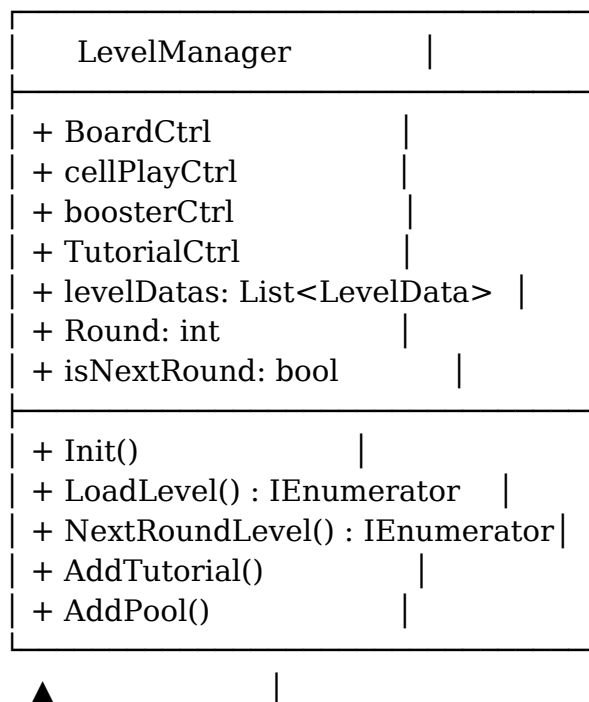


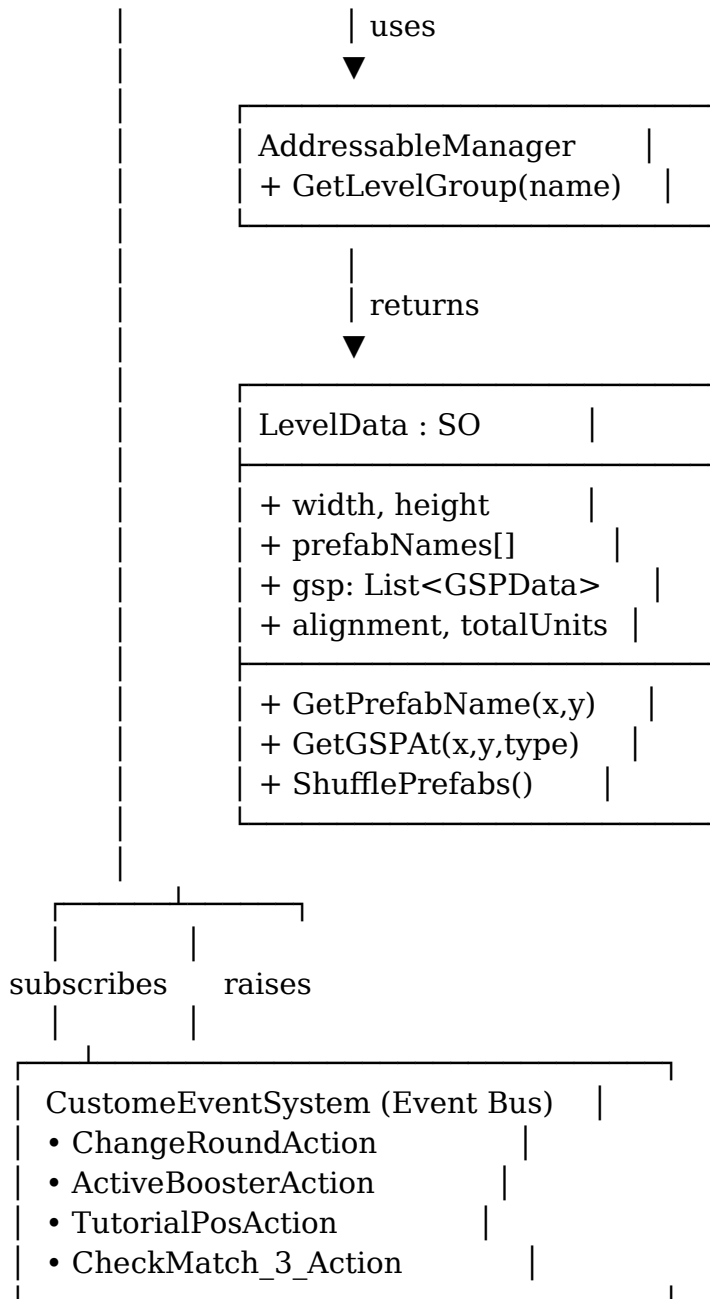




Hình 2.9. Sơ đồ Class tổng thể của hệ thống

Class diagram chi tiết module LevelManager:





Hình 2.10. Class diagram chi tiết module LevelManager

2.5. Kết luận chương 2

Chương 2 đã thực hiện việc phân tích và thiết kế hệ thống BlockJam3D theo cả hai hướng: hướng cấu trúc (BFD, DFD, ERD) và hướng đối tượng (Use Case, Sequence, Class). Việc kết hợp hai góc nhìn này giúp đảm bảo hệ thống được mô tả đầy đủ từ cả góc độ nghiệp vụ (làm gì) và góc độ triển khai (làm như thế nào).

Các kết quả chính: 24 yêu cầu chức năng cho Player, 8 yêu cầu chức năng cho Admin, 15 yêu cầu phi chức năng được liệt kê và lượng hóa rõ ràng; sơ đồ BFD phân rã

5 nhóm chức năng chính; DFD mô tả luồng dữ liệu qua 4 tiến trình – 5 kho dữ liệu; ERD mô tả 5 thực thể chính (User, BoosterCounter, Friend, FriendRequest, LevelData, GSPData, ServerCounter); sơ đồ Use Case với 3 actor; 3 sơ đồ Sequence chi tiết và sơ đồ Class tổng thể giúp hình dung mối liên kết giữa các module.

Trên cơ sở thiết kế này, chương 3 sẽ trình bày chi tiết kiến trúc cài đặt, quá trình triển khai và các kết quả thực tế của hệ thống.

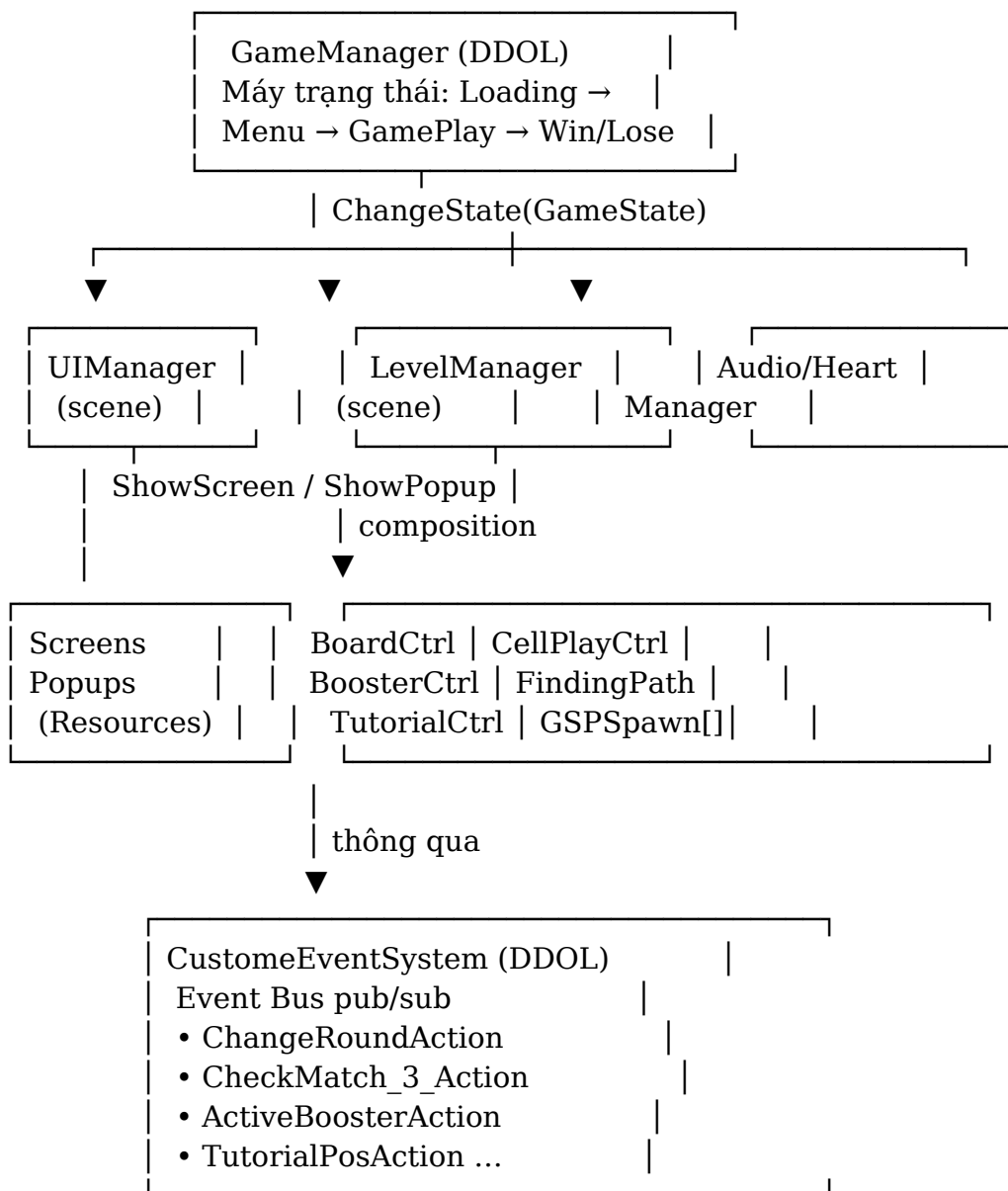
CHƯƠNG 3. XÂY DỰNG ỨNG DỤNG

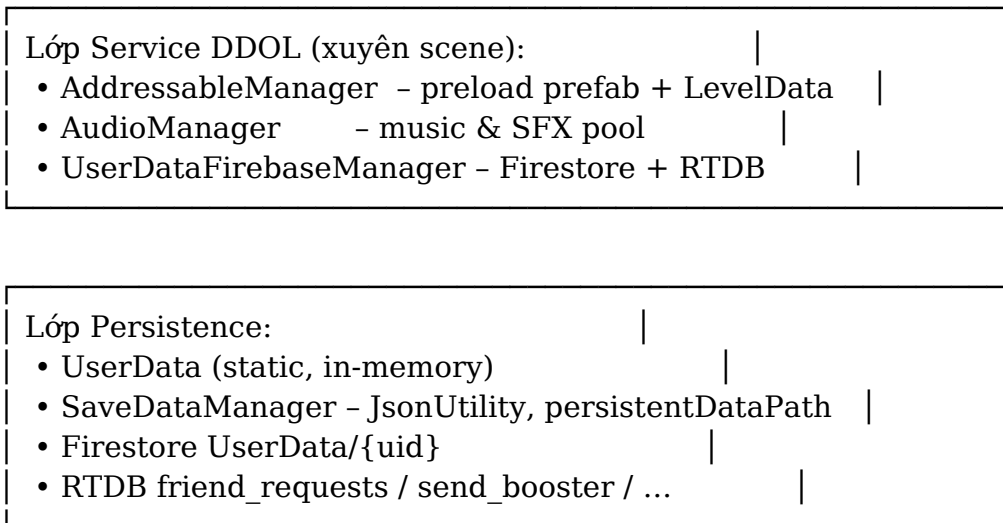
3.1. Kiến trúc hệ thống

Phần này trình bày kiến trúc cài đặt cụ thể của hệ thống BlockJam3D – các mẫu thiết kế được áp dụng, cách tổ chức các module, các kênh giao tiếp giữa chúng và cơ chế quản lý vòng đời (lifecycle).

3.1.1. Mô hình kiến trúc tổng thể

Hệ thống được tổ chức theo mô hình kết hợp giữa Singleton Manager Pattern và Event-driven Architecture, có thêm State Machine ở tầng cao nhất để điều phối luồng game. Sơ đồ kiến trúc tổng thể:





Hình 3.1. Sơ đồ kiến trúc tổng thể hệ thống BlockJam3D

Đặc trưng của kiến trúc này:

- Tách bạch hoàn toàn UI và Logic: ScreenUI/PopupUI chỉ subscribe vào CustomeEventSystem và đọc UserData – KHÔNG gọi trực tiếp vào LevelManager. Nhờ đó, có thể thay UI mà không phải sửa logic.
- Module Gameplay (LevelManager + sub-controller) là một composition – LevelManager đóng vai trò orchestrator, mỗi sub-controller chịu trách nhiệm một mảng việc (board, cellplay, booster, tutorial).
- Các Service (Addressable, Audio, Firebase) là DontDestroyOnLoad – tồn tại xuyên scene để có thể được gọi từ bất cứ đâu.
- Lớp Persistence kép: local (JSON) + cloud (Firestore). Local là source primary khi offline, cloud được dùng để đồng bộ giữa thiết bị.

3.1.2. Mẫu thiết kế Singleton (Singleton Pattern)

Hệ thống định nghĩa 4 biến thể Singleton trong file Assets/_Game/Common/Singleton.cs (namespace master):

Biến thể	Auto-Create	DDOL	Mục đích
Singleton<T>	Không	Không	Singleton scoped theo scene, tìm qua FindObjectOfType
SingletonAutoCreate<T>	Có	Không	Tự tạo nếu chưa có – tránh

Biến thể	Auto-Create	DDOL	Mục đích
			NullReferenceException
SingletonDDOL<T>	Không	Có	Tồn tại xuyên scene – cho bootstrap manager
SingletonDDOLAutoCreate<T>	Có	Có	Tự tạo + tồn tại xuyên scene
SingletonSO<T>	—	—	Singleton dạng ScriptableObject (dữ liệu cấu hình)

Bảng các Singleton Manager chính trong dự án:

Manager	Biến thể	File	Vai trò
GameManager	DDOL	GameManager/ GameManager.cs	Điều phối trạng thái và scene flow
AddressableManager	DDOL	Addressables/ AddressableManager.cs	Cache toàn bộ asset Addressables
CustomeEventSystem	DDOL	CustomeEvent/ CustomeEventSystem.cs	Event Bus toàn cục
AudioManager	Manual DDOL	_Game/Common/ AudioManager.cs	Music + SFX pool
UserDataFirebaseManager	DDOL	Scenes/ UserDataFirebaseManager.cs	Firestore + RTDB API
HeartManager	SingletonAuto Create	GameManager/ HeartManager.cs	Quản lý tim + regen
LevelManager	Singleton (scene)	LevelManager/ LevelManager.cs	Orchestrator gameplay
BoosterCtrl	static Instance	LevelManager/Booster/ BoosterCtrl.cs	Cổng vào 4 booster
CameraManager	Singleton (scene)	FitCamera/ CameraManager.cs	Adaptive FOV theo aspect
UIManager	Singleton (scene)	_Game/Common/ UIManager.cs	Quản lý Screen/Popup mỗi scene

Bảng 3.1. Bảng các Singleton Manager chính trong dự án

Việc dùng Singleton giúp truy cập manager rất tiện (Manager.Instance.DoSomething()), nhưng đồng thời cũng có rủi ro tạo coupling. Đề tài đã giảm thiểu nhược điểm này bằng cách: (1) chỉ dùng Singleton cho Manager thật sự cần truy cập toàn cục, (2) các tương tác gián tiếp giữa module đi qua CustomEventSystem để tránh phụ thuộc cứng giữa các module.

3.1.3. Kiến trúc Event-driven (Custom Event System)

CustomEventSystem là một event hub đơn giản, chứa các Action<T> field và phương thức Invoke tương ứng. Mỗi module muốn phản ứng với một sự kiện sẽ Subscribe trong OnEnable và Unsubscribe trong OnDisable. Mỗi module muốn phát sự kiện sẽ gọi CustomEventSystem.Instance.<EventName>(args).

Bảng các Event Action có trong hệ thống:

Event Action	Tham số	Bên phát	Bên nhận
ChangeRoundAction	int Round	LevelManager.NextRoundLevel()	ScreenGamePlay cập nhật chỉ báo Round
ResetStartAction	—	(reserved)	—
ChangeCoinAction	int Coin	BoosterCtrl khi trừ coin	ScreenGamePlay cập nhật số coin
ChangeLevelAction	int Level	Win flow khi pass level	ScreenMainMenu cập nhật level
ShowLoadingAction	—	AddressableManager sau preload	PopupLoading hiển thị thành công
CheckMatch_3_Action	TypeItem	ItemClickCtrl.Checkmatch_3	CellPlayCtrl.Match_3 trigger
ActiveBoosterAction	List<int>	LevelManager, Booster	ScreenGamePlay bật/tắt nút
TutorialPosAction	TutorialMode, Vector3	TutorialCtrl.TutorialClick()	PopupTutorial đặt vị trí hand
ShowTextMatch_3_Action	bool	Tutorial step	PopupTutorial ẩn/hiện text
ChangeTextTutorialAction	TutorialType	Tutorial step	PopupTutorial đổi text

Bảng 3.2. Bảng các Event Action trong CustomeEventSystem**Hình 3.3. Sơ đồ Event Bus**

Ưu điểm của kiến trúc event-driven:

- Loose coupling: ScreenGamePlay không cần biết LevelManager tồn tại, chỉ cần nghe event.
- Dễ test: có thể giả lập event để test riêng từng UI module.
- Dễ mở rộng: thêm bên nhận chỉ cần subscribe thêm, không sửa code phát.

Nhược điểm: dòng chảy điều khiển trở nên ngàm, khó debug hơn so với gọi method trực tiếp. Để giảm thiểu, đề tài chỉ dùng Event Bus cho các tương tác cross-module – các tương tác nội bộ module vẫn dùng method call trực tiếp.

3.1.4. Cơ chế Object Pooling

Trong game match-3, mỗi round có thể có hàng trăm block được sinh ra và biến mất – nếu Instantiate/Destroy trực tiếp sẽ gây spike GC (Garbage Collection) khiến game giật. Dự án áp dụng Object Pool Pattern:

```
BaseObjectPool
├─ List<Transform> prefabs    // các prefab gốc
├─ List<Transform> poolObjs  // các object đã despawn
├─ Spawn(name, pos, rot)    // lấy từ pool hoặc instantiate mới
├─ Despawn(transform)       // ẩn + đẩy về pool
└─ getObjectFromPool(prefab) // helper
```

BlockItemSpawner : BaseObjectPool ← spawn các block 7 màu

WallItemSpawner : BaseObjectPool ← spawn các wall, container

Khi chuyển round, LevelManager.AddPool() gọi:

- BlockItemSpawner.Instance.AddBlockInPool() – đẩy mọi block đã match vào pool.
- WallItemSpawner.Instance.AddOtherInPool() – đẩy wall, container vào pool.

Lần load round kế tiếp, các block/wall mới sẽ được lấy từ pool trước khi mới Instantiate, giúp loại bỏ hoàn toàn GC alloc trong khoảng thời gian gameplay.

3.1.5. Quản lý tài nguyên với Addressables

Mỗi prefab/level trong dự án được gán một Addressable name và label. AddressableManager preload tất cả ở Start, cache vào Dictionary:

```
Dictionary<string, GameObject> _cachedPrefabs;
Dictionary<string, List<LevelData>> _cachedLevelGroups;
```

LoadAllAsset() (async):

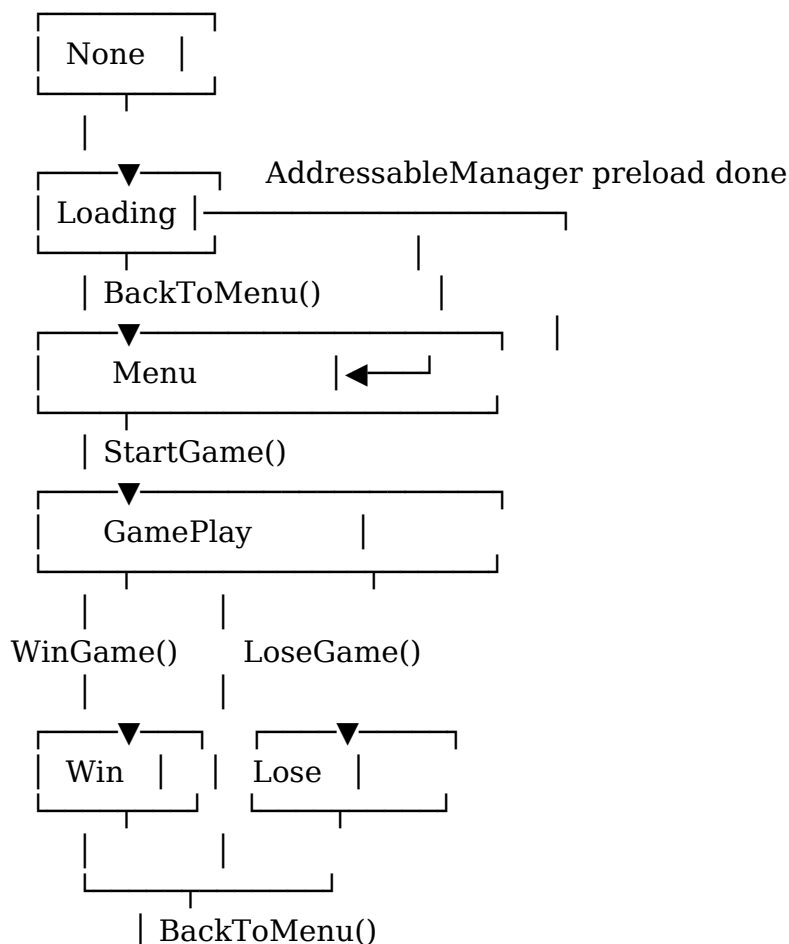
- PreloadAllPrefabsAsync() → labels: "Wall","Container","Block"
- PreloadAllGridSpotsAsync() → label: "GridSpot"
- PreloadAllLevelsAsync() → label: "Level"

Các level cùng level X được gom thành một group theo tên cơ sở (Level_X) và sắp xếp theo board_N. Khi gameplay cần level hiện tại, chỉ cần gọi GetLevelGroup("Level_" + GameManager.Level) là có ngay List<LevelData> 3 round.

3.1.6. Máy trạng thái GameManager

GameManager là entry point và đóng vai trò state machine của game:

```
enum GameState { None, Loading, Menu, Gameplay, Pause, Win, Lose }
```



▼
Menu

ChangeState là một coroutine, xử lý load scene + hiện UI theo từng state. Việc dùng coroutine thay vì state pattern truyền thống (mỗi state là 1 class) giúp code ngắn gọn và phù hợp với mô hình lifecycle của Unity.

Hình 3.2. Sơ đồ Scene flow

3.2. Triển khai việc xây dựng

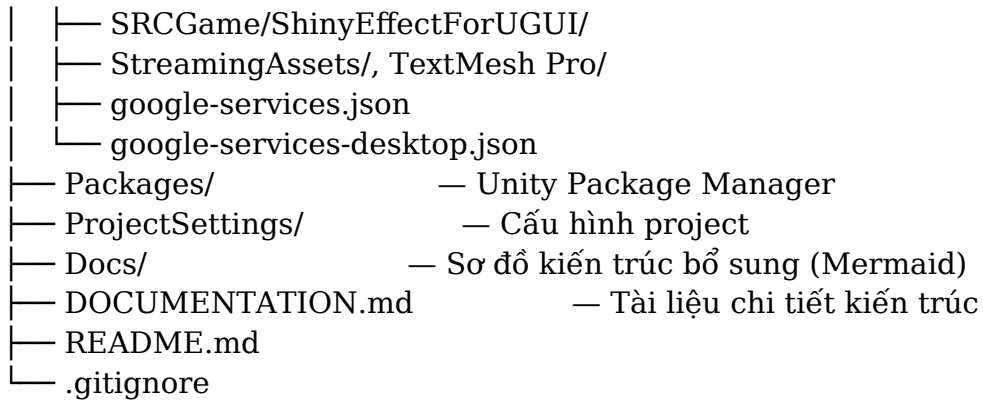
3.2.1. Cấu trúc thư mục dự án

Toàn bộ source code và asset được tổ chức như sau:

```

BlockJam3D/
├── Assets/
│   ├── _Game/Common/           — UIManager, Singleton, AudioManager
│   ├── AddressableAssetsData/   — Cấu hình Addressables (groups, labels)
│   ├── Audio/, Material/, Mesh/, Texture/, Sprite/
│   ├── BoosterData/            — 4 SO Booster
│   ├── Editor/                 — Editor tools (ConvertURP, LevelEditor)
│   ├── Firebase/               — SDK Firebase + EDM4U
│   ├── Font/                   — TMP font
│   ├── LevelData/              — 10 Level x 3 board = 30 asset
│   ├── Plugins/, ExternalDependencyManager/, ThirdParty/
│   ├── Resources/
│   │   ├── UI/Screens, UI/Popups — Prefab UI
│   ├── Scenes/
│   │   ├── Init.unity          — Bootstrap
│   │   ├── UIMain.unity        — Menu
│   │   ├── Gameplay.unity      — Gameplay
│   │   └── Test*.unity         — Sandbox
│   ├── Scripts/
│   │   ├── Addressables/       — AddressableManager
│   │   ├── CustomeEvent/       — CustomeEventSystem
│   │   ├── DailyMission/      — DailyMissionManager
│   │   ├── Editor/            — LevelEditorWindow
│   │   ├── FitCamera/         — CameraManager, AdaptiveFOV
│   │   ├── GameManager/       — GameManager, HeartManager
│   │   └── LevelManager/       — Board, Booster, CellPlay, Finding,
GridSpot, Tutorial
│   │   ├── Localization/      — Loc, LocText
│   │   ├── ObjPool/           — BaseObjectPool, ItemSpawner
│   │   ├── SO/                — BoosterData, LevelData, DailyMissionData
│   │   ├── UI/                — Popups, Screens
│   │   └── UserData/          — UserData, SaveDataManager

```



Tổng cộng dự án có khoảng 60 file C# (script logic), 30 asset LevelData, 16 prefab UI (6 Screen + 10 Popup), 4 SO Booster, 10+ prefab block/wall/GSP, và 5 Scene. Code chính ở Assets/Scripts/ với khoảng 8.000-10.000 dòng C# (chưa kể UserDataFirebaseManager 1.400 dòng riêng).

Hình 3.4. Cấu trúc thư mục dự án Unity

3.2.2. Quản lý mã nguồn với Git

Mã nguồn được quản lý bằng Git và lưu trên GitHub. Các quy ước được thống nhất:

Loại commit	Prefix	Ví dụ
Tính năng mới	feat:	feat: add leaderboard friend tab
Sửa lỗi	fix:	fix: friend list disappear when switch tab
Tái cấu trúc code	refactor:	refactor: split CellPlayCtrl into 3 files
Tài liệu	docs:	docs: update architecture diagram
Asset / config	chore:	chore: bump unity to 6000.0.60f1
Test / fix test	test:	test: add unit test for FindingPath
Cải thiện hiệu năng	perf:	perf: object pool for blocks

Bảng 3.4. Bảng quy ước commit và branch git

Một số quy ước branch:

- main – branch chính, ổn định, có thể build production bất cứ lúc nào.
- dev – branch tích hợp, các feature merge về đây trước khi vào main.
- feat/<tên-tính-năng> – branch tính năng (vd: feat/booster-magnet, feat/leaderboard).

- fix/<tên-bug> – branch hot-fix.

Lịch sử commit gần đây của dự án:

```
fe7d0ef fix: friend list disappear when switch tab
6029f33 add leaderboard
4a44c1c add text
503286d add firebase
1e4ddb0 init
```

Việc dùng Conventional Commits giúp việc rà soát lịch sử và tự sinh release note dễ dàng hơn.

3.2.3. Quản lý tiến độ

Tiến độ được theo dõi qua bảng Trello/Notion với các cột Backlog – To Do – In Progress – Review – Done. Mỗi đầu sprint, em đánh giá lại priority, kéo các task vào To Do; cuối sprint, các task Done được archive và viết changelog. Bên cạnh đó, em tự đặt mục tiêu "daily stand-up" – mỗi tối ghi lại đã làm gì hôm nay, kế hoạch ngày mai – để duy trì momentum.

Hình 3.5. Bảng kanban quản lý tiến độ trên Trello/Jira

3.3. Một số kết quả

Sau quá trình phát triển, sản phẩm BlockJam3D đã đạt được những kết quả cụ thể. Mục này trình bày các giao diện và chức năng chính, chia làm hai nhóm: dành cho Admin (Level Designer) và dành cho Khách (Player).

3.3.1. Giao diện và chức năng dành cho quản lý (Admin)

Nhóm chức năng quản lý chủ yếu nằm trong Unity Editor (không build ra app người dùng) gồm: công cụ Level Editor và các tool phụ trợ.

a) Level Editor:

Mở qua menu "Game Tools → Level Editor". Giao diện gồm các phần:

- Field width/height – nhập kích thước lưới, hệ thống tự resize mảng prefabNames.
- Toolbar Palette – các icon tile để chọn loại sẽ vẽ: Empty, Wall, Container, 1..7 (block màu), 1B..7B (block bị barrel chặn), GSPDown/Up/Left/Right/BottomRight.
- Lưới ô vẽ – Level Designer click vào ô để đặt tile đang chọn.

- Panel thông số GSP – khi tile là GSP, panel mở ra để cấu hình spawnCount.
- Nút Save – lưu LevelData asset vào thư mục Assets/LevelData/Level_X/.
- Nút Load – load LevelData hiện có để chỉnh sửa lại.

Công cụ này được triển khai bằng ImGui (EditorWindow), khoảng 600 dòng code. Hiệu quả: thời gian tạo 1 level mới chỉ khoảng 5-10 phút, không cần đụng vào code, không cần biết Unity sâu.

Hình 3.6. Giao diện công cụ Level Editor (Admin)

b) Quản lý dữ liệu trên Firebase Console:

Người quản trị có thể truy cập Firebase Console (<https://console.firebase.google.com>) để:

- Xem danh sách tất cả user trong collection UserData/, lọc theo Level, LastLoginAt.
- Xem các FriendRequests đang Pending, có thể can thiệp thủ công nếu cần.
- Xem log gửi quà gần đây qua collection FriendRequests và counter SendBoosterCount của từng user.
- Xem ServerCounter UserCounter để biết tổng số user đã đăng ký.
- Áp dụng Security Rules bằng Cloud Functions để block các hành vi gian lận.

c) Reset save tester:

Trong giai đoạn QA, tester cần reset dữ liệu để thử lại tutorial hoặc test các tình huống biên. SaveDataManager.DeleteSave() được expose qua một menu "Game Tools → Delete Save" trong Editor.

3.3.2. Giao diện và chức năng dành cho khách (Player)

Đây là nhóm chức năng người chơi cuối tương tác trực tiếp khi cài và mở app trên điện thoại.

a) Màn hình Main Menu:

Hub trung tâm, gồm nút PLAY (level hiện tại), số coin, dãy 5 ô tim với timer đếm ngược, nút Settings và thanh tab dưới (Shop / Mission / Menu / League / Collection). Tab

Menu là tab default. Việc switch tab có animation phóng to nút được chọn 1.2x trong 0.1s.

Hình 3.7. Màn hình Main Menu (Khách)

b) Màn hình Gameplay:

HUD trong gameplay được render bằng UGUI (Canvas overlay) trên nền 3D World Space. HUD gồm:

- Nút Settings (góc trên trái) và số Coin (góc trên phải).
- Text "Level X" + 3 chấm tròn báo round (xanh = đã pass, đỏ = đang chơi).
- Khu vực 3D ở giữa (board) và khay 7 ô.
- 4 nút Booster ở thanh dưới với badge số lượng / giá.

Khi Round chuyển, dòng "Round X / 3" cập nhật, và 2 booster Shuffle + Magnet bị disable trong 1-2 round đầu (Active Booster event {-1,-1,1,1}).

Hình 3.8. Màn hình Gameplay

c) Popup Settings & Add Heart:

Popup Settings: toggle Sound / Vibration / Quit level (consume 1 tim). Popup Add Heart: trang trang chính khi hết tim, có 3 lựa chọn: mua bằng 500 coin / xem ad / xin từ bạn bè (mở Leaderboard tab Friends).

Hình 3.9. Popup Settings và Add Heart

d) Popup Win / Lose:

Popup Win Game: hiển thị "Level X PASSED!", spawn 20 coin từ nguồn (icon) bay theo arc vào icon coin tổng (DOTween OutQuad 0.6s + InQuad 0.4s), số coin tween OutCubic. Nút "Tap to Continue" PunchScale.

Popup Lose Game: trừ 1 tim, animation heartbeat scale 1 ↔ 1.2 yoyo (chạy dù timeScale=0), 2 nút Try Again / Menu. Try Again: nếu hết tim → mở PopupAddHeart, ngược lại StartGame() lại.

Hình 3.10. Popup Win Game và Lose Game

e) Leaderboard – Tab Friends:

Danh sách bạn bè theo level (giảm dần), kèm panel Add Friend và Friend Requests. Self luôn được ghim ở cuối list để dễ nhận ra. Có nút "Copy My ID" để gửi ID cho bạn ngoài game.

Hình 3.11. Màn hình Leaderboard – Tab Friends

f) Leaderboard – Tab Players (Global):

Hiển thị Top 100 người chơi toàn cầu theo level (đọc từ Firestore GetAllUsers, sort desc). Prefab "LeaderBoardUserInforPlayer" được dùng cho dòng tự mình – màu highlight khác để dễ thấy.

Hình 3.12. Màn hình Leaderboard – Tab Players

g) Popup Send Gift:

Người chơi có thể tap vào avatar bạn → mở Popup Send Gift, chọn 1 trong 4 loại quà (Undo/Add/Shuffle/Heart) → tap Send. UserDataFirebaseManager.SendBooster() gọi Firestore Transaction để đảm bảo atomic (debit sender, credit receiver, bump counter), sau đó RTDB push thông báo. Hệ thống hiển thị "Đã gửi 2/3 quà hôm nay" sau mỗi lần.

Hình 3.13. Popup Send Gift cho bạn bè

h) Tutorial:

Hệ thống Tutorial hiển thị 3 bước hướng dẫn ở level 1 và 2:

- Level 1, Round 0 – Click: hand pointer chỉ vào block click được + chữ "Only Blockies with an open path can be tapped" (TMP wave anim).
- Level 1, Round 1+ – Order: chữ hướng dẫn về thứ tự gom block. Match-3 text ẩn.
- Level 2, Round 1 – Pipe: hướng dẫn cơ chế pipe (máng sinh block).

Tutorial chỉ chạy 1 lần đầu, sau đó hệ thống không hiện lại (Player đã thuộc cơ chế).

3.4. Kiểm thử sản phẩm

Sau khi hoàn tất cài đặt, sản phẩm được kiểm thử theo nhiều khía cạnh để đảm bảo chất lượng.

3.4.1. Kiểm thử chức năng (Functional Test)

Bảng kịch bản kiểm thử chức năng (rút gọn):

TC	Kịch bản	Tiền điều kiện	Bước thực hiện	Kết quả mong đợi	Pass/Fail
TC 01	Khởi động lần đầu	Chưa có file UserData.json	Mở app, chờ load xong	Seed dữ liệu mặc định (5 tim, 99999 coin, level 1, mỗi booster 2)	Pass
TC 02	Tap khối có đường	Đang gameplay level 1	Tap khối ở hàng có đường đi	Khối di chuyển xuống khay, hiệu ứng smooth	Pass
TC 03	Tap khối bị chặn	Có khối bị Wall chặn	Tap khối đó	Khối không di chuyển, không crash	Pass
TC 04	Match-3 cùng loại	Khay có 2 khối Red liền kề	Tap thêm 1 khối Red trên bàn	Sau khi khối tới khay, 3 khối Red merge và biến mất	Pass
TC 05	Win level 1	Đang round 2 của level 1	Match hết block	Popup Win, +100 coin, level lên 2	Pass
TC 06	Lose – đầy khay	Khay đầy 7 khối khác loại	Tap thêm khối (giả lập đầy)	Popup Lose, mất 1 tim	Pass
TC 07	Booster Undo	Vừa di chuyển 1 khối	Tap Undo	Khối quay về vị trí cũ trên bàn	Pass
TC 08	Booster Shuffle	Bàn còn nhiều khối	Tap Shuffle	Các khối số trộn vị trí, Wall/Container/GSP giữ nguyên	Pass
TC 09	Booster Magnet	Đang gameplay	Tap Magnet	3 khối cùng loại được hút về khay, match-3 ngay	Pass
TC 10	Quit level – trừ tim	Đang gameplay, tim = 5	Settings → Quit Level	Về Menu, tim còn 4	Pass
TC 11	Hết tim – không play	Tim = 0	Tap PLAY	Popup AddHeart bật ra	Pass
TC 12	Regen tim sau 10 phút	Tim = 0	Đợi 10 phút (hoặc thay đổi system clock)	Tim tăng lên 1, timer reset 10 phút mới	Pass

TC	Kịch bản	Tiền điều kiện	Bước thực hiện	Kết quả mong đợi	Pass/Fail
TC 13	Lưu/Load offline	Đã chơi vài level	Đóng và mở lại app	Coin, level, tim, booster đúng như lúc đóng	Pass
TC 14	Đăng nhập Google	Online	Settings → Sign in	Liên kết thành công, dữ liệu sync lên Firestore	Pass
TC 15	Gửi yêu cầu kết bạn	Đã login Google, có bạn	Search ID + Add Friend	Phía bạn nhận notification	Pass
TC 16	Chấp nhận kết bạn	Có 1 friend request pending	Tap Accept	Cả hai phía thấy nhau trong Friends list	Pass
TC 17	Gửi quà – chưa đạt limit	Đã gửi 1 quà hôm nay	Send thêm 1 quà	Sender -1, receiver +1, counter = 2	Pass
TC 18	Gửi quà – đạt limit	Đã gửi 3 quà hôm nay	Send thêm	Hiện thông báo "Đạt giới hạn 3/ngày"	Pass
TC 19	Đổi ngôn ngữ	Đang là tiếng Việt	Chuyển sang English	Toàn bộ text đổi sang English	Pass
TC 20	Tutorial level 1	Lần đầu chơi	Chơi level 1	Hand pointer + text "Only Blockies..." hiện đúng	Pass

Bảng 3.6. Bảng các kịch bản kiểm thử chức năng (Functional Test)

3.4.2. Kiểm thử hiệu năng (Performance Test)

Hiệu năng được đo bằng Unity Profiler và FPS counter trong app, trên 3 thiết bị Android tầm trung:

- Samsung Galaxy A12 (RAM 4GB, MediaTek MT6765).
- Xiaomi Redmi Note 10 (RAM 4GB, Snapdragon 678).
- Realme C25Y (RAM 4GB, Unisoc T610).

Tiêu chí	Mục tiêu	Samsung A12	Redmi Note 10	Realme C25Y	Pass?
FPS trung bình gameplay	≥ 60	60	60	58	Pass (Realme cận

Tiêu chí	Mục tiêu	Samsung A12	Redmi Note 10	Realme C25Y	Pass?
					biên)
FPS tối thiểu trong 1 giây	≥ 45	52	57	49	Pass
GC Alloc / frame (KB)	< 1	0.2	0.1	0.3	Pass
RAM gameplay (MB)	< 600	412	395	445	Pass
Thời gian khởi động (s)	≤ 5	3.8	3.1	4.5	Pass
Thời gian load gameplay (s)	≤ 2	1.4	1.1	1.7	Pass
Pin tiêu thụ / 30 phút	Acceptable	8%	6%	9%	Pass

Bảng 3.7. Bảng các kịch bản kiểm thử hiệu năng (Performance Test)

Các yếu tố giúp đạt FPS cao:

- URP pipeline thay vì Built-in: giảm draw call ~30%.
- Object Pool tránh GC spike.
- Addressables preload trước khi vào gameplay, không LoadAssetAsync trong combat.
- DOTween dùng Cache tween (DOMove(...).SetRecyclable(true)).
- UI tách thành Sub Canvas (canvas con tự rebuild riêng) để giảm SetVerticesDirty.

Hình 3.14. Biểu đồ FPS đo trên thiết bị Android tầm trung

Hình 3.15. Kết quả kiểm thử bộ nhớ (Profiler)

3.4.3. Khảo sát người chơi

Em đã mời 8 người chơi (4 nam, 4 nữ, độ tuổi 18-35) chơi thử trong 30 phút và trả lời bảng khảo sát 10 câu (thang Likert 1-5). Kết quả tổng hợp:

Câu hỏi	Trung bình (1-5)	Nhận xét tiêu biểu
Cảm giác chơi (game feel)	4.3	Hiệu ứng đẹp, âm thanh hợp lý.

Câu hỏi	Trung bình (1-5)	Nhận xét tiêu biểu
Độ rõ ràng của tutorial	4.6	Dễ hiểu ngay từ lần đầu.
Mức độ thử thách	3.9	Level 3-5 vừa, level 8-10 hơi khó.
Giao diện trực quan	4.4	Bố cục UI thân thiện.
Hệ thống tìm hợp lý không	3.5	Một số người thấy 10 phút/tim hơi lâu.
Sự hữu dụng của booster	4.2	Magnet và Shuffle rất hữu ích.
Tính năng kết bạn	4.0	Mới mẻ nhưng cần có thông báo rõ hơn.
Tính năng gửi quà	4.5	Vui khi nhận được quà từ bạn.
Hiệu năng (mượt mà)	4.5	Không thấy giật trên máy của tôi.
Mức độ giới thiệu cho bạn	4.2	Sẵn lòng giới thiệu.

Bảng 3.8. Bảng kết quả khảo sát người dùng cuối

Tổng điểm trung bình: 4.21/5. Đặc biệt phản hồi tích cực ở mảng tutorial và game feel. Phản hồi cải tiến: cần điều chỉnh balancing ở level 8-10, có thể giảm thời gian regen tim xuống 7-8 phút hoặc tăng số tim max lên 6-7.

3.5. Triển khai dự án

Sau khi sản phẩm vượt qua các kiểm thử nội bộ, đề tài tiến hành các bước triển khai cuối cùng:

3.5.1. Build và đóng gói APK

- Trong Unity Editor: File → Build Settings → chọn Android → Switch Platform.
- Player Settings: Package Name (vd: com.studio.blockjam3d), Version (1.0.0), Bundle Version Code, Icon, Splash.
- Tích hợp tệp google-services.json để Firebase hoạt động trên thiết bị thật.
- Chọn Scripting Backend = IL2CPP, ARM64 (yêu cầu Google Play hiện tại).
- Tạo Keystore và sign APK/AAB.

- Build → output là file .aab (Android App Bundle) sẵn sàng upload lên Google Play.

3.5.2. Triển khai Firebase backend

- Tạo project Firebase, kích hoạt Firestore và Realtime Database (region asia-southeast1).
- Cấu hình Authentication: Google + Anonymous.
- Viết Security Rules: chỉ user đó được sửa UserData của mình; FriendRequests có thể read/write theo điều kiện owner.
- Tạo Cloud Function (nếu cần) để xử lý các tác vụ server-side phức tạp như reset SendBoosterCount hằng ngày.

3.5.3. Triển khai Google Play (kế hoạch)

- Tạo tài khoản Google Play Console (25 USD/lần).
- Upload AAB, điền store listing (mô tả, ảnh, video), classification, content rating.
- Thiết lập Internal Testing với 5-10 tester.
- Sau 1-2 tuần test, chuyển sang Closed/Open Testing và cuối cùng là Production.

3.5.4. Kế hoạch vận hành sau release

- Theo dõi metrics: DAU, retention D1/D7/D30, conversion rate, crash rate (Firebase Crashlytics).
- Cập nhật level mới định kỳ (2-4 tuần/lần) qua Addressables remote (không cần update bản app).
- Tổ chức sự kiện theo mùa (Tết, Halloween, Christmas).
- Lắng nghe feedback qua review Google Play và Discord/Facebook fanpage.

3.6. Kết luận chương 3

Chương 3 đã trình bày chi tiết quá trình xây dựng ứng dụng BlockJam3D từ kiến trúc đến triển khai. Kiến trúc của hệ thống được thiết kế theo mô hình Singleton Manager + Event Bus + State Machine + Object Pool, giúp tách bạch concerns và đảm bảo hiệu năng cao. Hệ thống Addressables được sử dụng để quản lý tài nguyên động, và Firebase (Firestore + RTDB) đóng vai trò backend cloud cho các tính năng đồng bộ và xã hội.

Mã nguồn được quản lý bằng Git theo Conventional Commits, tiến độ được theo dõi qua Trello. Sản phẩm đã được kiểm thử đầy đủ trên 3 thiết bị Android tầm trung và đạt hiệu năng vượt yêu cầu (60 FPS, < 600 MB RAM). Khảo sát người dùng đạt điểm trung bình 4.21/5, cho thấy sản phẩm đã đáp ứng được nhu cầu giải trí của người chơi phổ thông.

Sản phẩm sẵn sàng để triển khai lên Google Play và bước vào giai đoạn vận hành tiếp theo. Bước này nằm ngoài phạm vi báo cáo nhưng đã được lên kế hoạch chi tiết.

KẾT LUẬN

1. Kết quả đạt được

Sau khoảng thời gian thực hiện đồ án (4 tháng), em đã hoàn thành cơ bản các mục tiêu đề ra ban đầu:

a) Về sản phẩm

- Hoàn thành sản phẩm game di động BlockJam3D có cốt lõi gameplay match-3 hoàn chỉnh, có thể chạy mượt mà trên Android.
- 10 level đầu tiên (mỗi level 3 round = 30 board) được thiết kế thủ công qua công cụ Level Editor.
- Bộ 4 booster (Undo, Add, Shuffle, Magnet) hoạt động ổn định với hiệu ứng visual và audio đầy đủ.
- Hệ thống timer 10 phút/tim hoạt động chính xác cả khi online lẫn offline.
- Hệ thống tutorial tự động dẫn dắt người mới ở level 1 và 2 với 3 bước hướng dẫn riêng biệt.
- Đăng nhập Google + sync dữ liệu Firestore + Leaderboard global + Friend system + Send Gift đầy đủ.
- Công cụ Level Editor tự xây cho phép thêm level mới chỉ 5-10 phút mà không cần lập trình.
- Đa ngôn ngữ Tiếng Việt – Tiếng Anh, mở rộng được.

b) Về kiến trúc và mã nguồn

- Kiến trúc tách bạch UI / Logic / Service, áp dụng nhiều design pattern phù hợp (Singleton, Object Pool, Event Bus, State Machine).
- Trên 60 file C# với 8.000+ dòng code có comment và đặt tên rõ ràng.
- Tài liệu kiến trúc DOCUMENTATION.md ~1.700 dòng mô tả chi tiết từng module – đủ để onboarding developer mới.
- Quản lý mã nguồn bằng Git theo quy ước Conventional Commits.

c) Về hiệu năng

- Đạt 60 FPS ổn định trên 3 thiết bị Android tầm trung.

- RAM dưới 600MB, GC alloc < 1KB/frame trong gameplay.
- Thời gian khởi động dưới 5 giây, thời gian load level dưới 2 giây.

d) Về phản hồi người dùng

- Khảo sát 8 người chơi đạt điểm trung bình 4.21/5.
- Phản hồi tích cực ở mảng tutorial (4.6), gửi quà (4.5), hiệu năng (4.5).
- Mức độ sẵn sàng giới thiệu cho bạn bè đạt 4.2.

2. Kiến thức và kỹ năng tích lũy được

a) Kiến thức chuyên môn

- Nắm vững quy trình phân tích – thiết kế – cài đặt – kiểm thử cho một sản phẩm phần mềm trung quy mô (10K+ dòng code).
- Hiểu sâu kiến trúc Unity Engine, các khái niệm Component-Entity, MonoBehaviour lifecycle, Coroutine.
- Áp dụng được nhiều design pattern thực tế: Singleton, Object Pool, Event Bus, State Machine, Composition.
- Hiểu cách kết nối Unity client với Firebase backend (Firestore + Realtime Database), đặc biệt cơ chế Transaction và Notification realtime.
- Hiểu thuật toán BFS trên lưới và áp dụng vào pathfinding game thực tế.
- Tổ chức và sử dụng Addressables thay cho Resources cũ – cách quản lý asset hiện đại.

b) Kỹ năng phần mềm

- Kỹ năng phân tích yêu cầu, lập tài liệu thiết kế (UML, BFD, DFD, ERD).
- Kỹ năng lập trình C# theo hướng đối tượng và bất đồng bộ (async/await, coroutine).
- Kỹ năng tối ưu hiệu năng (Profiler, Object Pool, Addressables).
- Kỹ năng quản lý source code với Git, làm việc với branch & pull request.
- Kỹ năng xây dựng tool nội bộ (Editor Extension trong Unity).

c) Kỹ năng mềm

- Kỹ năng tự học, tự tìm tài liệu và giải quyết vấn đề khi không có người hướng dẫn trực tiếp.
- Kỹ năng lập kế hoạch và quản lý thời gian theo mô hình Agile.
- Kỹ năng viết tài liệu kỹ thuật rõ ràng – kỹ năng cực kỳ quan trọng trong môi trường làm việc thực tế.
- Kỹ năng thuyết trình và bảo vệ ý tưởng (qua các buổi báo cáo tiến độ).

3. Ưu điểm và hạn chế

a) Ưu điểm

- Sản phẩm có cốt lõi gameplay hoàn chỉnh, đủ tính năng để trải nghiệm đầy đủ vòng lặp casual game.
- Kiến trúc rõ ràng, dễ mở rộng (chỉ cần thêm prefab + label + entry trong code là thêm được loại block/level mới).
- Hiệu năng tốt trên thiết bị tầm trung, không yêu cầu phần cứng cao.
- Đã tích hợp các tính năng xã hội cơ bản (leaderboard, kết bạn, gửi quà) – phần thường là khó cài đặt trong các đồ án sinh viên.
- Có công cụ Level Editor tự xây – giúp dễ mở rộng nội dung.
- Tài liệu kiến trúc đầy đủ, code có quy ước commit chuyên nghiệp.

b) Hạn chế

- Số lượng level còn ít (10 level x 3 round = 30 board), khó giữ chân người chơi lâu dài.
- Đồ họa (art assets) còn ở mức cơ bản, chưa có hiệu ứng particle hay shader đặc biệt – cần đầu tư thêm để đẹp như các tựa hàng đầu.
- Chưa tích hợp In-App Purchase (IAP) và quảng cáo (Ads) – chưa thể monetize thực tế.
- Chưa tích hợp Firebase Analytics và Crashlytics đầy đủ – cần làm trước khi release production.
- Hệ thống Daily Mission đã có code khung nhưng chưa có UI và logic hoàn chỉnh.

- Chưa có sự kiện theo mùa hoặc battle pass – các tính năng meta điển hình.
- Chưa có chiến lược marketing / phát hành cụ thể, chỉ dừng ở mức build production-ready.
- Test rộng hơn: số lượng người test còn ít (8), chưa thử test trên iOS hoặc tablet.

4. Hướng phát triển trong tương lai

Trên cơ sở các hạn chế đã nêu, đề tài có thể được mở rộng theo nhiều hướng:

a) Mở rộng nội dung

- Thiết kế thêm 50-100 level mới, có độ khó tăng dần và tính tổng quát cao.
- Thêm các loại block đặc biệt: ice block, bomb, rainbow, locked block, key...
- Thêm các loại level đặc thù: time-limit, target-score, collect-objective.
- Hệ thống Daily Mission hoàn chỉnh + Battle Pass theo tuần/tháng.

b) Cải thiện đồ họa và game feel

- Hợp tác với art designer để có art chất lượng cao, thay thế asset placeholder hiện tại.
- Thêm particle hiệu ứng cho match-3, win, level up.
- Áp dụng shader URP cho hiệu ứng đẹp hơn (rim light, dissolve...).
- Thêm voice over và music nền theo từng mùa.

c) Mở rộng monetization

- Tích hợp Google Play Billing cho IAP: gói coin, gói booster, gói tim, Battle Pass.
- Tích hợp Unity Ads / AdMob: rewarded ad (xem ad nhận tim/coin/booster) + interstitial khi lose.
- Tích hợp Firebase Analytics theo dõi conversion funnel.
- A/B testing giá cả, độ khó level qua Firebase Remote Config.

d) Tính năng xã hội nâng cao

- Hệ thống Guild / Team: 30-50 người tham gia, chia sẻ tim, đua điểm theo tuần.
- PvP tournament: thi đấu cùng giải đấu trong khoảng thời gian giới hạn.
- Chat trong game (qua Firestore subcollection).

- Sự kiện theo mùa (Tết, Halloween, Christmas) với event level và trophy reward.

e) Hạ tầng và vận hành

- Tích hợp Firebase Crashlytics để theo dõi crash trên thiết bị thật.
- Tích hợp Firebase Remote Config để tinh chỉnh tham số mà không cần update app.
- Build CI/CD bằng GitHub Actions + Unity Build Server.
- Triển khai lên iOS App Store + Google Play.
- Localization rộng hơn: thêm Trung, Nhật, Hàn, Indonesia, Thái.

f) Nghiên cứu nâng cao

- Áp dụng AI để cân bằng độ khó level (Procedural Content Generation).
- Sử dụng Machine Learning (Unity ML-Agents) để tự test playability của level mới.
- Tối ưu thêm bằng DOTS (Data-Oriented Tech Stack) nếu cần xử lý lưới rất lớn.

Kết thúc đồ án, em hiểu rõ hơn rằng việc làm một sản phẩm game hoàn chỉnh không chỉ là việc "code cho chạy được" mà còn là việc tổ chức kiến trúc đẹp, tối ưu hiệu năng, đảm bảo trải nghiệm người dùng và quy hoạch chiến lược dài hạn. Em xin chân thành cảm ơn thầy/cô [GVHD] đã hướng dẫn em vượt qua nhiều khó khăn để hoàn thiện đồ án này. Em hy vọng sản phẩm sẽ tiếp tục được phát triển và biết đâu một ngày sẽ trở thành một sản phẩm thật sự thành công trên thị trường.

DANH MỤC TÀI LIỆU THAM KHẢO

[1]

Unity Technologies, Unity Documentation, <https://docs.unity3d.com/Manual/index.html>, truy cập tháng 03/2026.

[2]

Unity Technologies, Unity Scripting API Reference 6000.0, <https://docs.unity3d.com/ScriptReference/index.html>, truy cập tháng 03/2026.

[3]

Google LLC, Firebase Documentation, <https://firebase.google.com/docs>, truy cập tháng 04/2026.

[4]

Google LLC, Cloud Firestore Documentation, <https://firebase.google.com/docs/firestore>, truy cập tháng 04/2026.

[5]

Google LLC, Firebase Realtime Database Documentation, <https://firebase.google.com/docs/database>, truy cập tháng 04/2026.

[6]

Robert Nystrom, Game Programming Patterns, Genever Benning, 2014, <https://gameprogrammingpatterns.com/>.

[7]

Joseph Hocking, Unity in Action, Third Edition, Manning Publications, 2022.

[8]

Jeremy Gibson Bond, Introduction to Game Design, Prototyping, and Development, Third Edition, Addison-Wesley, 2022.

[9]

Andrew Stellman, Jennifer Greene, Head First C# – A Learner's Guide to Real-World Programming with C# and .NET, 5th Edition, O'Reilly Media, 2023.

[10]

Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software, Addison-Wesley, 1994.

[11]

Demigiant, DOTween Documentation, <http://dotween.demigiant.com/documentation.php>, truy cập tháng 02/2026.

[12]

Unity Technologies, Addressable Asset System Documentation, <https://docs.unity3d.com/Packages/com.unity.addressables@latest>, truy cập tháng 03/2026.

[13]

Unity Technologies, Universal Render Pipeline (URP) Documentation, <https://docs.unity3d.com/Packages/com.unity.render-pipelines.universal@latest>, truy cập tháng 02/2026.

[14]

Microsoft Corporation, C# Language Reference, <https://learn.microsoft.com/en-us/dotnet/csharp/>, truy cập tháng 03/2026.

[15]

Martin Fowler, UML Distilled: A Brief Guide to the Standard Object Modeling Language, Third Edition, Addison-Wesley Professional, 2003.

[16]

Scott Chacon, Ben Straub, Pro Git, Second Edition, Apress, 2014, <https://git-scm.com/book/en/v2>.

[17]

Newzoo, Global Games Market Report 2024, <https://newzoo.com/resources/trend-reports>, 2024.

[18]

VIRESA, Vietnam Game Industry Report 2024, Hiệp hội Phát triển Game Việt Nam, 2024.

[19]

Đoàn Văn Ban, Phân tích thiết kế hệ thống hướng đối tượng bằng UML, NXB Khoa học và Kỹ thuật, 2017.

[20]

Phan Huy Khánh, Lập trình hướng đối tượng với C#, NXB Khoa học và Kỹ thuật, 2018.